

Way Back Home - Live Bidirectional Multi-Agent system

來源：<https://codelabs.developers.google.com/way-back-home-level-4/instructions?hl=zh-tw>

步驟數：6

在本程式碼研究室中，您將建構及部署即時雙向多代理系統，完成模擬太空救援任務。您將使用 Google Agent Development Kit (ADK) 建立主要代理，用於即時語音和視訊互動，並將藍圖擷取作業委派給連線至 Redis 資料庫的專用代理。這項功能的主要特色是主動式安全監控，以串流工具的形式實作，可持續分析影片串流中的視覺危害，並立即觸發警報。接著，我們會將以 React 前端和 FastAPI/WebSocket 後端建構的完整全端應用程式部署至 Google Cloud Run，展示建立及運作精密的互動式 AI 系統的完整工作流程。

目錄

1. [1. 使命](#)
2. [2. 設定環境](#)
3. [3. 在 Redis 中建構架構儲存空間，並使用 ADK 建構雙向代理](#)
4. [4. 將 BIDI-Streams 代理連線至遠端代理和串流工具](#)
5. [5. 深入瞭解即時多模態事件串流](#)
6. [6. 部署至正式環境 \(選用\)](#)

步驟 1 · 約 0 分鐘

1. 使命



你漂流在無聲無息、未知的浩瀚宇宙中。巨大的太陽脈衝撕裂了你的船隻，將你困在宇宙的某個角落，這裡沒有任何星圖。

經過多日艱辛的維修，引擎終於恢復熟悉的嗡嗡聲。火箭已可運作。你甚至還設法與母艦建立了遠距上行鏈路。即將出發。你已準備好回家。

但正當你準備啟動跳躍引擎時，一陣靜電干擾中傳來求救訊號。感應器偵測到來自「奧西曼迪亞斯」星球的求救訊號。倖存者被困在這個瀕臨滅亡的世界，他們的船隻也擱淺了。你的任務至關重要：在星球大氣層崩解前救出他們。

他們唯一的逃生方法是使用**外星科技**打造的古老廢棄火箭。雖然火箭可以運作，但**曲速引擎**已損壞。如要拯救倖存者，您必須遠端連線至他們的**揮發性工作台**，並手動組裝更換用的硬碟。

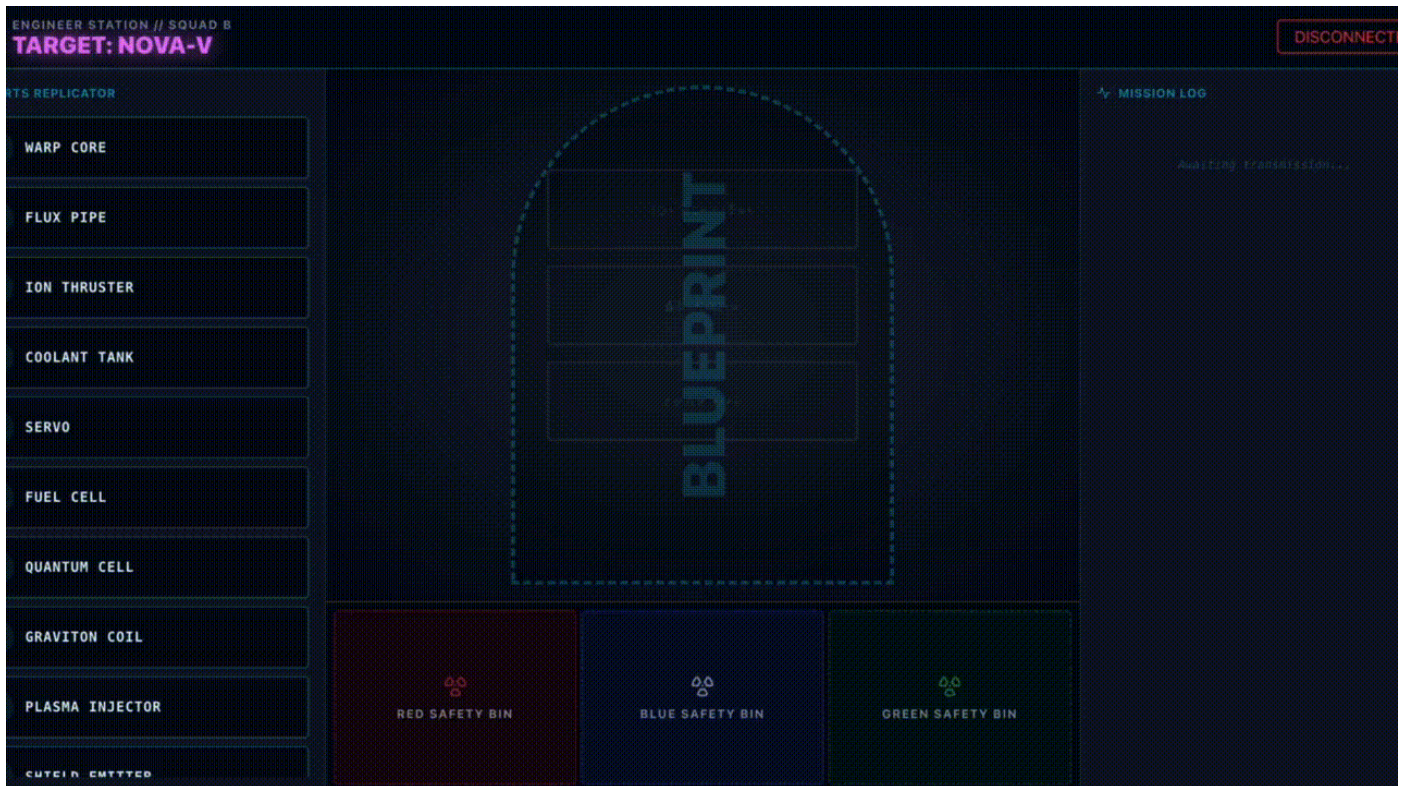
挑戰

您對這種外星科技一無所知，而且這種科技非常脆弱。不穩定的元件可能在幾秒內變成放射性危害。您有一次操作 Volatile Workbench 的機會。目前的 AI 助理無法同時處理視覺資料和技術手冊，導致指令出現幻覺，並錯過危險警告。

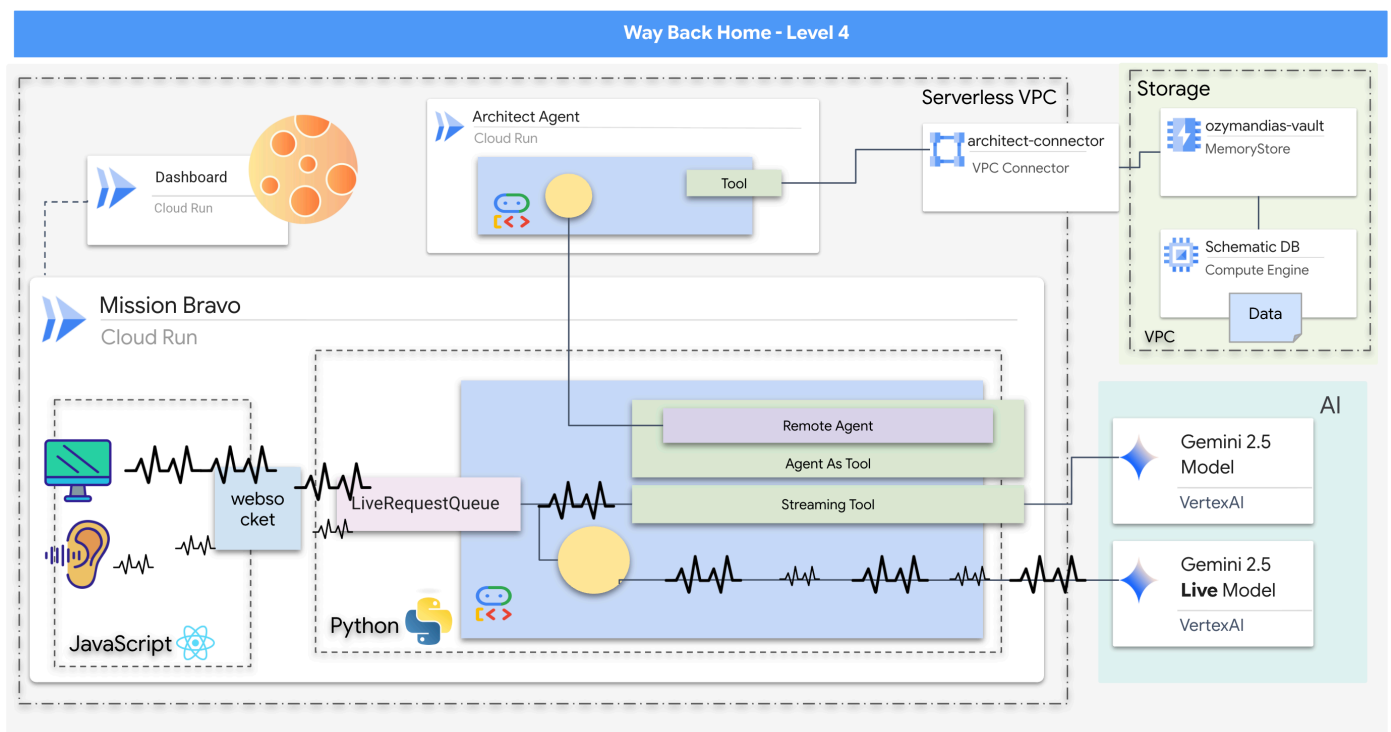
如要成功，您必須將 AI 從單一實體升級為協作式**多代理系統**。

您的任務目標：

按照新多代理系統提供的即時專屬指示組裝曲速引擎。



建構目標



- 這套雙向多代理 AI 系統可即時運作，並設有中央派遣代理，負責管理使用者互動及協調專業代理。
- 架構代理程式：連線至 Redis 資料庫，以擷取及提供架構資料。
- 主動式安全監控功能，可使用串流工具分析即時影像動態饋給中的視覺危害，並觸發即時快訊。
- 以 **React** 為基礎的前端，提供與系統互動的使用者介面，並將影片和音訊串流傳輸至後端代理程式。

學習目標

技術 / 概念	說明
Google Agent Development Kit (ADK)	您將使用 ADK 建構、測試及管理代理，並運用其架構處理即時通訊、工具整合和代理生命週期。
雙向 (Bidi) 串流	您將實作雙向串流服務專員，實現自然、低延遲的雙向通訊，讓人類和 AI 都能即時中斷和回應。
多代理系統	您將瞭解如何設計分散式 AI 系統，讓主要代理將工作分派給專業代理，實現工作區隔，並打造更具擴充性的架構。
Agent2Agent (A2A) 通訊協定	您將使用 A2A 通訊協定，啟用 Dispatch Agent 和 Architect Agent 之間的通訊，讓兩者可以互相探索功能並交換資料。
串流工具	您將實作串流工具，做為背景程序持續分析影片動態饋給，監控狀態變化 (危險)，並主動產生結果。
Google Cloud Run 和 Memorystore	您將使用 Cloud Run 代管代理程式服務，並以 Memorystore (Redis) 做為永久資料庫，將整個多代理程式應用程式部署到正式環境。
FastAPI 和 WebSocket	後端是使用 FastAPI 和 WebSocket 建構而成，可處理串流音訊、影片和代理程式回覆所需的高效能即時通訊。
React 前端	您將使用以 React 為基礎的前端，擷取及串流傳輸使用者媒體 (音訊/影片)，並顯示 AI 代理的即時回覆。

步驟 2 · 約 0 分鐘

2. 設定環境

info

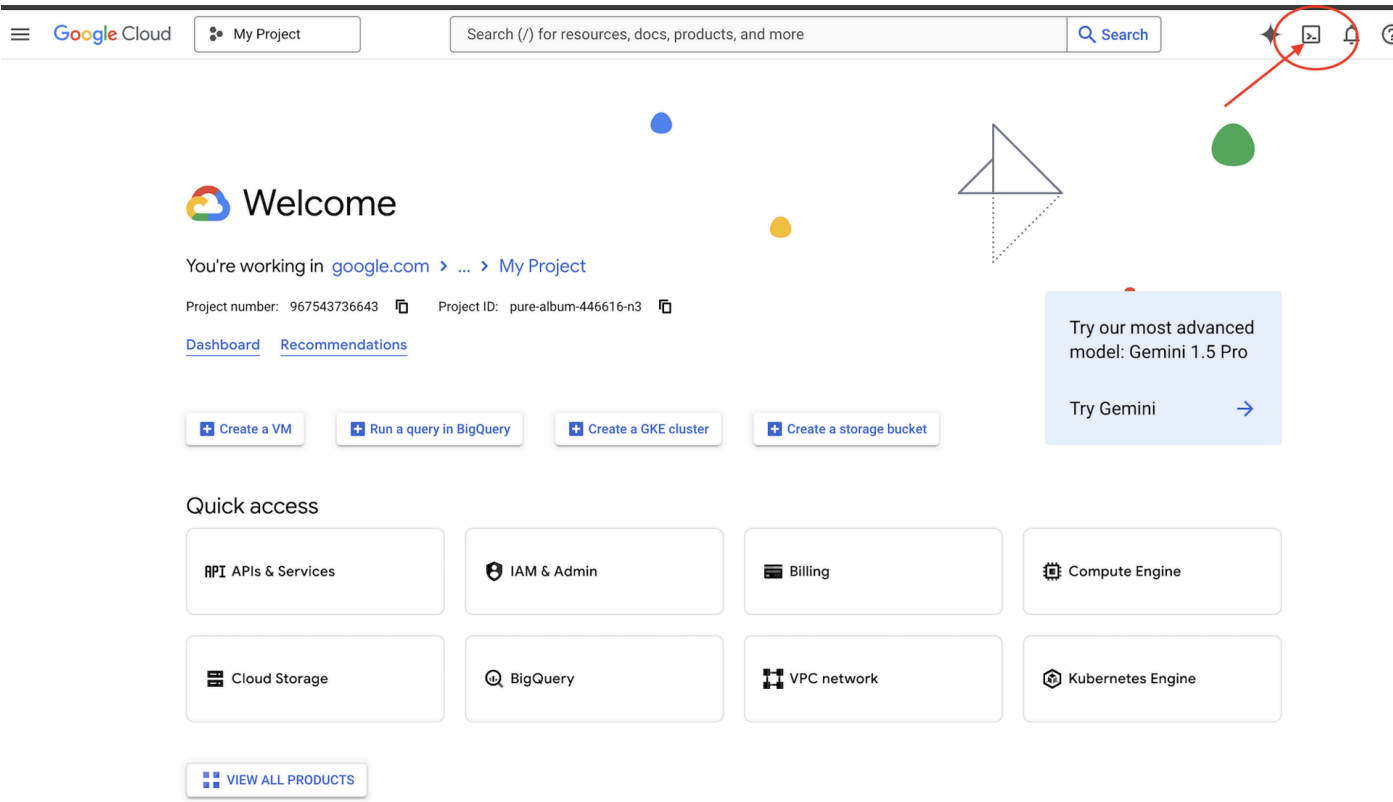
從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

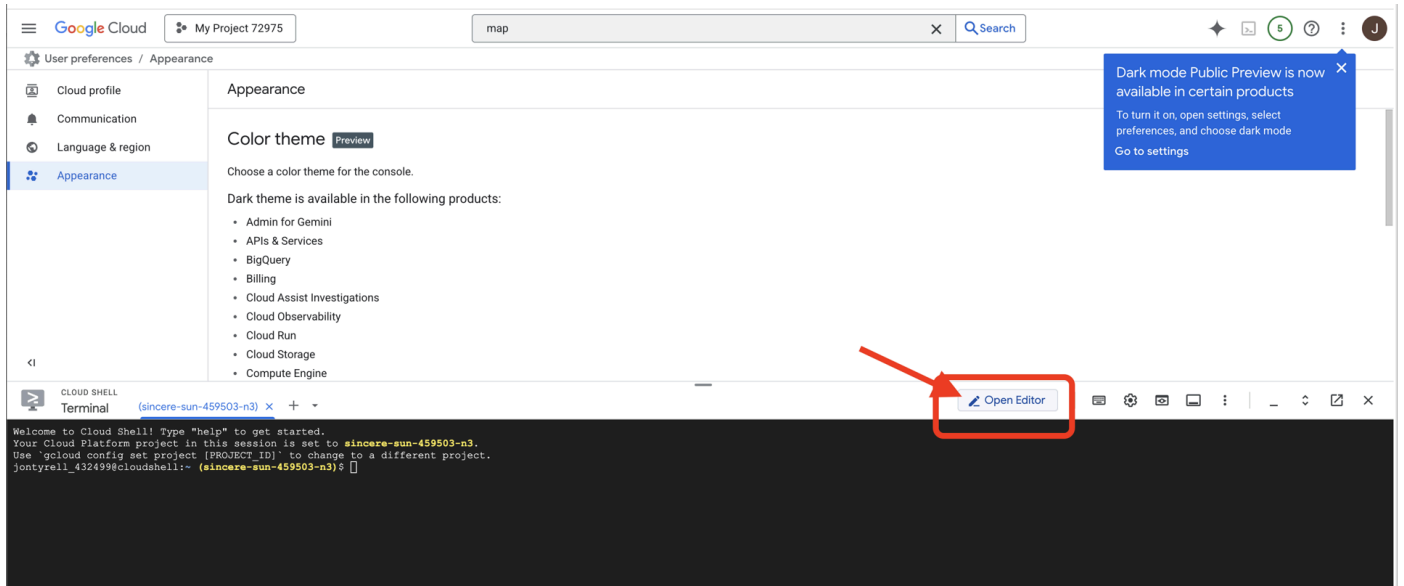
啟用

存取 Cloud Shell

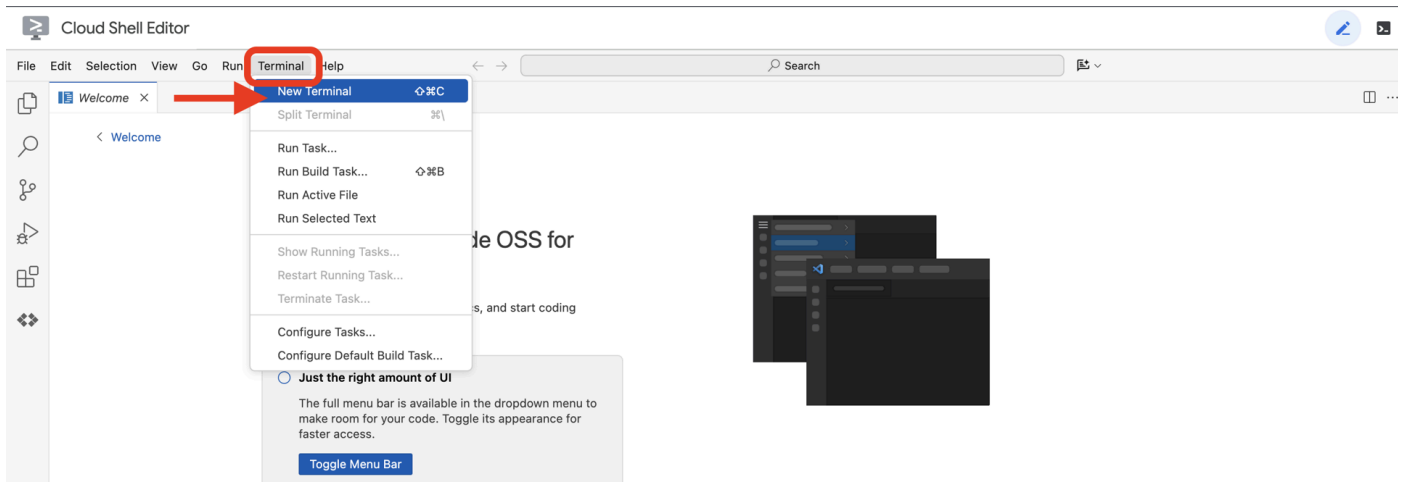
👉 點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」(這是 Cloud Shell 窗格頂端的終端機形狀圖示)



👉 按一下「Open Editor」按鈕 (類似於有鉛筆的開啟資料夾)。視窗中會開啟 Cloud Shell 程式碼編輯器。左側會顯示檔案總管。



👉 在雲端 IDE 中開啟終端機，



👉 在終端機中，使用下列指令驗證您是否已通過驗證，以及專案是否已設為您的專案 ID：

```
gcloud auth list
```

您的帳戶應該會顯示為 (ACTIVE)。

必要條件

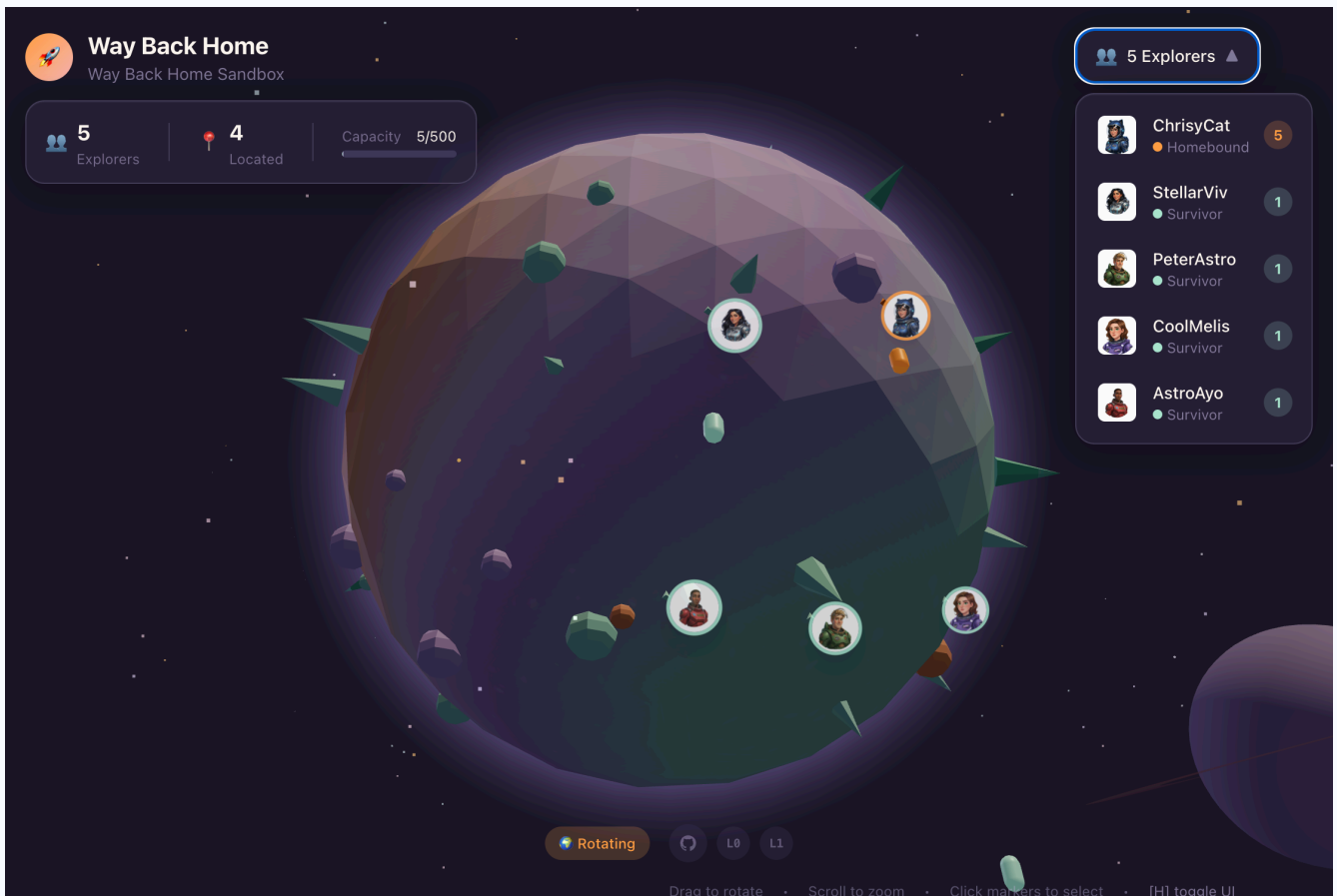
第 0 級為選用 (但建議使用)

您可以在 0 級完成這項任務，但建議先完成 0 級，享受更身歷其境的體驗，在完成任務的過程中，看著信號在世界地圖上亮起。

i 如果你已完成第 0 級，請驗證：

[按這裡開始第 0 級](#)

- [] `config.json` 在專案根目錄中，並提供參與者 ID 和座標
- [] 你的虛擬人偶會顯示在世界地圖上
- [] 座標處顯示 (昏暗) 信標
- [] 您已在工作目錄中複製 `way-back-home` 資料夾



執行準備指令碼，將工作區切換至新的任務參數。

```
cd $HOME/way-back-home/level_4
chmod +x scripts/level_0_complete_prep.sh
./scripts/level_0_complete_prep.sh
```

指令碼會嘗試偵測有效的 Google Cloud 設定。

- 重複使用第 0 級專案：如果偵測到的專案 ID 正確無誤，只要按下 **Enter** 鍵即可。
- 切換專案：輸入要使用的新專案 ID，然後按下 **Enter** 鍵。

i 如果您略過第 0 級，則需要佈建新的環境：

請按照這些步驟設定必要基礎架構和 Google Cloud 專案。

複製並初始化

在終端機中執行下列指令，下載任務檔案並啟動設定指令碼：

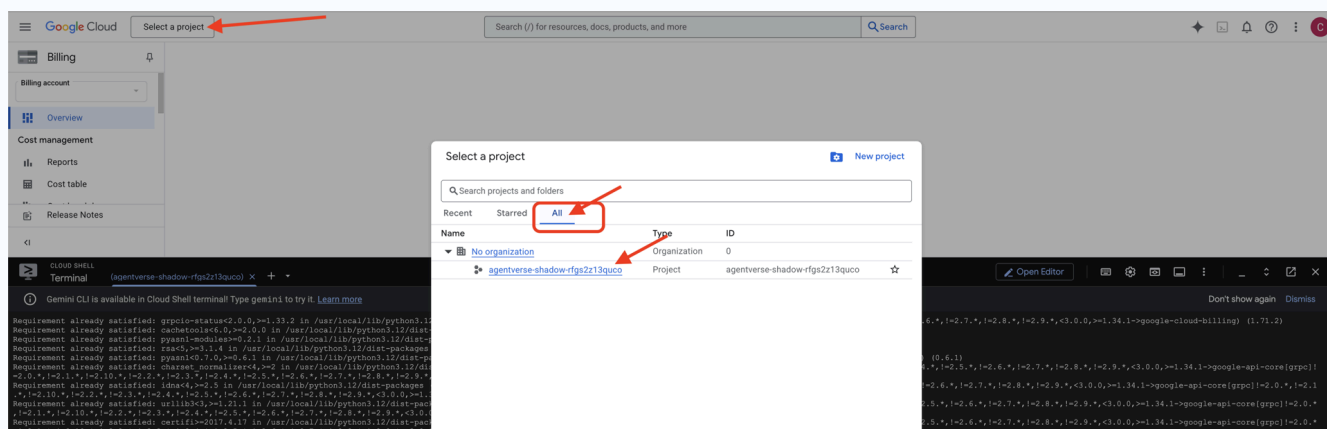
```
git clone https://github.com/gca-americas/way-back-home.git
cd $HOME/way-back-home/level_4/scripts/
chmod +x init.sh
. init.sh
```

指令碼會處理 Google Cloud 專案的建立作業和基本設定。

確認專案選項

重要步驟：即使指令碼執行完畢，Google Cloud 控制台可能仍會查看舊專案或預設專案。你必須手動切換：

1. 前往 console.cloud.google.com。
2. 按一下頁面頂端的「專案選取器」下拉式選單。
3. 按一下「全部」分頁標籤 (新專案可能尚未顯示在「最近」中)。
4. 選取指令碼剛建立的「專案 ID」。



設定專案環境

返回終端機，設定有效專案並啟用必要的 Google Cloud 服務 (Cloud Run、Vertex AI 等)，完成設定。

在終端機中設定專案 ID：

```
gcloud config set project $(cat ~/project_id.txt) --quiet
```

啟用必要服務：

```
gcloud services enable compute.googleapis.com \
  artifactregistry.googleapis.com \
  run.googleapis.com \
  cloudbuild.googleapis.com \
  iam.googleapis.com \
  aiplatform.googleapis.com \
  cloudresourcemanager.googleapis.com \
  redis.googleapis.com \
  vpcaccess.googleapis.com
```

安裝依附元件

👉🏠 前往第 4 級，然後安裝必要的 Python 套件：

```
cd $HOME/way-back-home/level_4
uv sync
```

主要依附元件如下：

套件	目的
<code>fastapi</code>	高效能網頁架構，適用於衛星電台和 SSE 串流
<code>uvicorn</code>	執行 FastAPI 應用程式所需的 ASGI 伺服器
<code>google-adk</code>	用於建構 Formation Agent 的 Agent Development Kit
<code>a2a-sdk</code>	Agent2Agent 通訊協定程式庫，可進行標準化通訊
<code>google-genai</code>	用來存取 Gemini 模型的原生用戶端
<code>redis</code>	用於連線至 Schematic Vault (Memorystore) 的 Python 用戶端
<code>websockets</code>	支援即時雙向通訊
<code>python-dotenv</code>	管理環境變數和設定密鑰
<code>pydantic</code>	資料驗證和設定管理

驗證設定

開始撰寫程式碼前，請先確認所有系統都正常運作。執行驗證指令碼，稽核 Google Cloud 專案、API 和 Python 依附元件。

👉 執行驗證指令碼：

```
cd $HOME/way-back-home/level_4/scripts
chmod +x verify_setup.sh
. verify_setup.sh
```

👀 你應該會看到一連串的綠色勾號 (✅)。

- 如果看到紅十字 (❌)，請按照輸出內容中的建議修正指令 (例如 `gcloud services enable ...` 或 `pip install ...`) 操作。
- 附註：目前 `.env` 的黃色警告訊息可以接受，我們會在下一個步驟中建立該檔案。

🚀 Verifying Mission Bravo (Level 4) Infrastructure...

✅ Google Cloud Project: xxxxxxxx

✅ Cloud APIs: Active

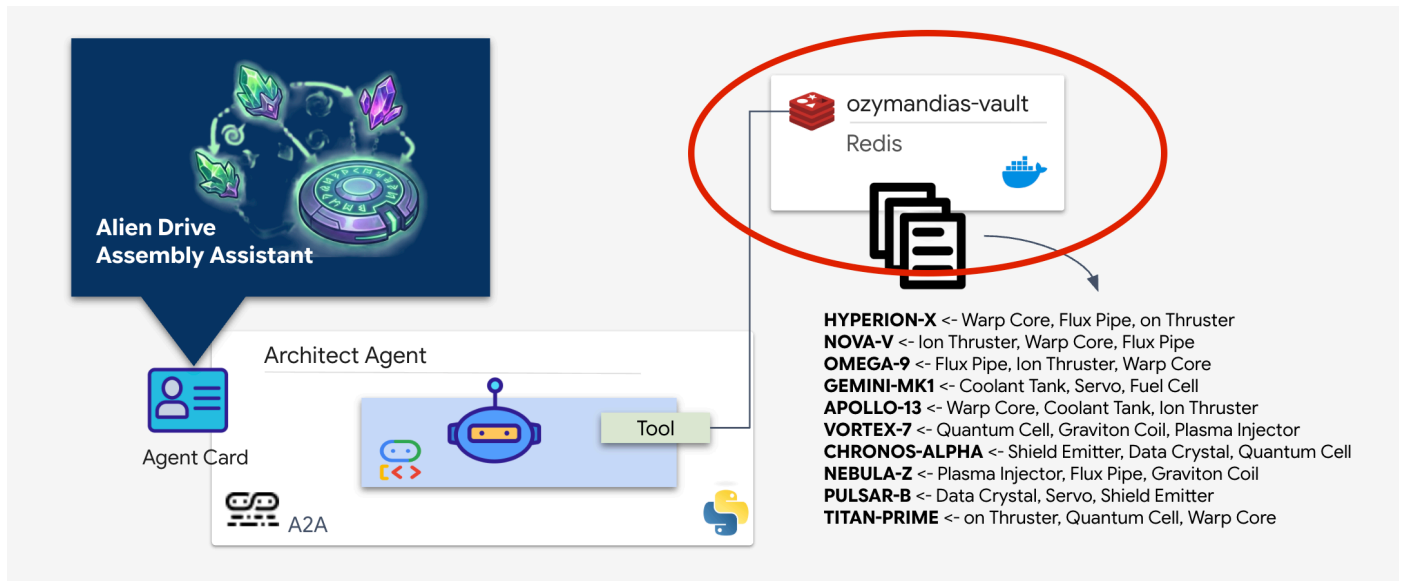
✅ Python Environment: Ready

🎉 SYSTEMS ONLINE. READY FOR MISSION.

步驟 3 · 約 0 分鐘

3. 在 Redis 中建構架構儲存空間，並使用 ADK 建構雙向代理

您已找到行星示意圖存放區，其中包含廢棄火箭的藍圖。如要準確擷取這項資料，您必須與存放區的專屬管理介面 (即 Architect 代理) 互動。



佈建 Schematic Vault (Redis)

在架構師提供協助前，我們必須確保資料託管在安全的高可用性環境中。我們將使用 **Redis** 做為外星人示意圖的快速資料儲存庫。為方便開發，我們將啟動本機 Redis 執行個體，但稍後會提供如何使用 Google Cloud Memorystore 部署至正式環境的操作說明。

👉 在終端機中執行下列指令，佈建 Redis 執行個體 (這可能需要 2 到 3 分鐘)：

```
docker run -d --name ozymandias-vault -p 6379:6379 redis:8.6-rc1-alpine
```

👉 如要載入初步資料，請執行下列指令來進入 Redis Shell：

```
docker exec -it ozymandias-vault redis-cli
```

(提示會變更為 127.0.0.1:6379)

👉 將下列指令貼到裡面：

```
RPUSH "HYPERION-X" "Warp Core" "Flux Pipe" "Ion Thruster"  
RPUSH "NOVA-V" "Ion Thruster" "Warp Core" "Flux Pipe"  
RPUSH "OMEGA-9" "Flux Pipe" "Ion Thruster" "Warp Core"  
RPUSH "GEMINI-MK1" "Coolant Tank" "Servo" "Fuel Cell"  
RPUSH "APOLLO-13" "Warp Core" "Coolant Tank" "Ion Thruster"  
RPUSH "VORTEX-7" "Quantum Cell" "Graviton Coil" "Plasma Injector"  
RPUSH "CHRONOS-ALPHA" "Shield Emitter" "Data Crystal" "Quantum Cell"  
RPUSH "NEBULA-Z" "Plasma Injector" "Flux Pipe" "Graviton Coil"  
RPUSH "PULSAR-B" "Data Crystal" "Servo" "Shield Emitter"  
RPUSH "TITAN-PRIME" "Ion Thruster" "Quantum Cell" "Warp Core"
```

👉 輸入 `exit` 即可返回一般 Shell。

👉 如要直接從終端機查詢特定貨運，確認資料是否存在，請執行：

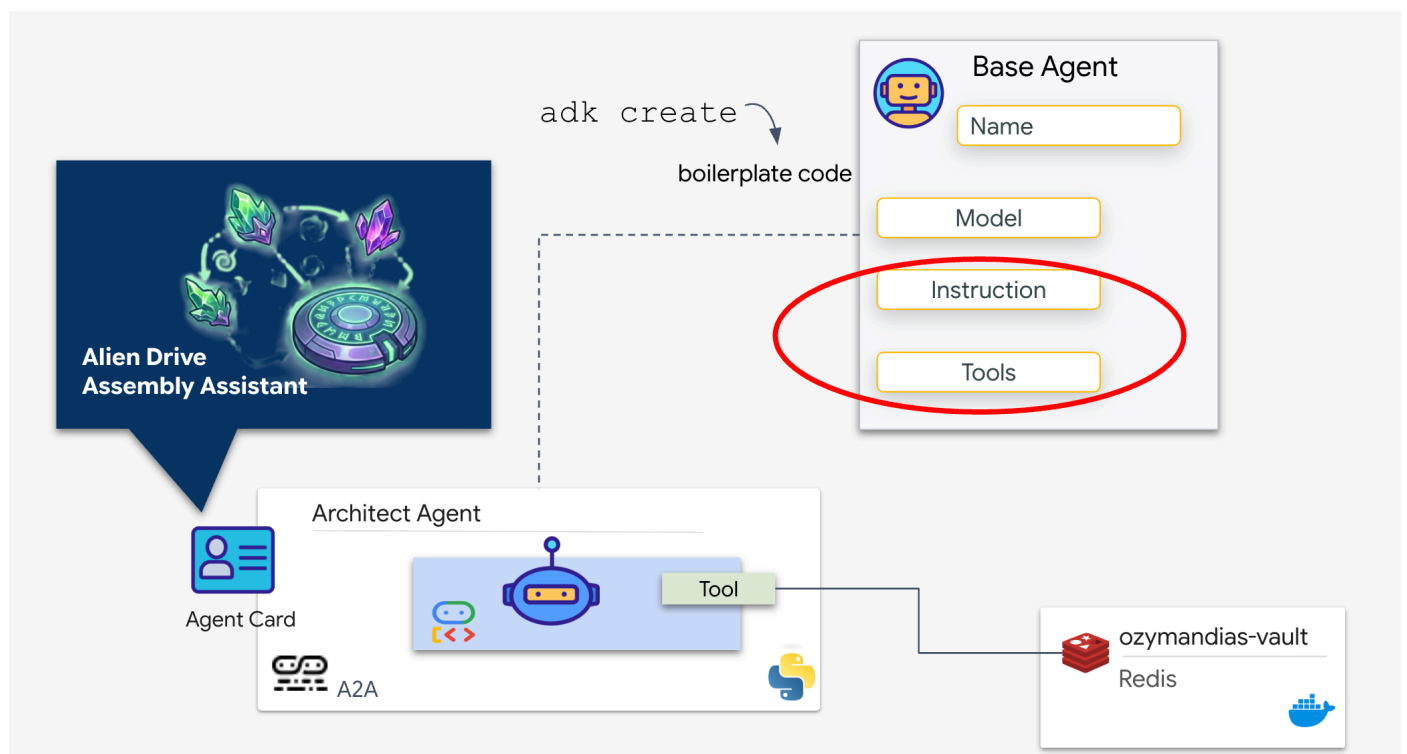
```
# Check 'TITAN-PRIME'
docker exec ozymandias-vault redis-cli LRange "TITAN-PRIME" 0 -1
```

👁️ 預期的輸出內容如下：

```
1) "Ion Thruster"
2) "Quantum Cell"
3) "Warp Core"
```

實作 Architect Agent

架構代理程式是專門的代理程式，負責從 Redis 保管庫擷取示意圖藍圖。這項功能可做為專屬資料介面，確保主要派遣代理程式收到準確的結構化資訊，不必瞭解基礎資料庫邏輯。



Google Agent Development Kit (ADK) 是模組化框架，可實現這種多代理設定。可處理兩個重要層級：

1. **連線和工作階段生命週期**：與即時 API 互動需要複雜的通訊協定管理，包括處理信號交換、驗證和 keep-alive 信號。
2. **函式呼叫**：這是「模型-程式碼-模型往返」。當 LLM 判斷需要資料時，會輸出結構化函式呼叫。ADK 會攔截這項要求、執行 Python 程式碼 (`lookup_schematic_tool`)，並在幾毫秒內將結果回饋至模型的环境。

現在要建構 **Architect**。這個服務專員無法存取相機。這個函式僅用於接收「磁碟機名稱」，並從資料庫傳回「零件清單」。

👉 我們將使用 `adk create` 指令。這是 Agent Development Kit (ADK) 的工具，可自動生成新代理的樣板程式碼和檔案結構，節省設定時間。

```
cd $HOME/way-back-home/level_4/backend/  
uv run adk create architect_agent
```

設定代理程式

CLI 會啟動互動式設定精靈。請使用下列回覆設定代理程式：

1. **選擇模型**：選取「選項 1」 (Gemini Flash)。
 - 注意：具體版本 (例如 2.5、3.0) 可能因供應情形而異。請一律選擇「Flash」變體，以提高速度。
2. **選擇後端**：選取「選項 2」 (Vertex AI)。
3. **輸入 Google Cloud 專案 ID**：按下 **Enter** 鍵接受預設值 (從環境中偵測到)。
4. **輸入 Google Cloud 區域**：按下 **Enter** 鍵接受預設值 (`us-central1`)。

👁️ 終端機互動應如下所示：

```
(way-back-home) user@cloudshell:~/way-back-home/level_4/agent$ adk create  
architect_agent  
  
Choose a model for the root agent:  
1. gemini-2.5-flash  
2. Other models (fill later)  
Choose model (1, 2): 1  
  
1. Google AI  
2. Vertex AI  
Choose a backend (1, 2): 2  
  
You need an existing Google Cloud account and project...  
Enter Google Cloud project ID [your-project-id]: <PRESS ENTER>  
Enter Google Cloud region [us-central1]: <PRESS ENTER>  
  
Agent created in /home/user/way-back-home/level_4/agent/architect_agent:  
- .env  
- __init__.py  
- agent.py
```

這時畫面上應該會顯示 `Agent created` 成功訊息。這會產生骨架程式碼，我們將在下一個步驟中修改。

👉 在編輯器中找到並開啟新建立的 `$HOME/way-back-home/level_4/backend/architect_agent/agent.py` 檔案。在第一個匯入行之後，將工具程式碼片段新增至檔案：

```

import os
import redis

REDIS_IP = os.environ.get('REDIS_HOST', 'localhost')
r = redis.Redis(host=REDIS_IP, port=6379, decode_responses=True)

def lookup_schematic_tool(drive_name: str) -> list[str]:
    """Returns the ordered list of parts for a drive from local Redis."""

    # Logic to clean input like "TARGET: X" -> "X"
    clean_name = drive_name.replace("TARGET:", "").replace("TARGET", "").strip()
    clean_name = clean_name.replace(":", "").strip()

    # LRange gets all items in the list (index 0 to -1)
    result = r.lrange(clean_name, 0, -1)

    if not result:
        print(f"[ARCHITECT] Error: Drive ID '{clean_name}' not found in Redis.")
        return ["ERROR: Drive ID not found."]

    print(f"[ARCHITECT] Returning schematic for {clean_name}: {result}")
    return result

```

✍️ 將 `root_agent` 定義中的整行 **instruction** 換成下列內容，並加入我們先前定義的工具：

```

instruction='''SYSTEM ROLE: Database API.
INPUT: Text string (Drive Name).
TASK: Run `lookup_schematic_tool`.
OUTPUT: Return ONLY the raw list from the tool.
CONSTRAINT: Do NOT add conversational text.
''',
tools=[lookup_schematic_tool],

```

ADK 的優勢

有了 Architect 線上版，我們現在有了可靠的資料來源。在將這個代理程式連結至主要代理程式之前，Agent Development Kit (ADK) 可簡化建構及測試 AI 代理程式的複雜性，因此具有顯著優勢。透過內建的 `adk web` 開發人員控制台，我們可以先隔離並驗證 Architect Agent 的功能 (尤其是工具呼叫功能)，再將其整合至較大的多代理系統。這種模組化開發和測試方法，對於建構功能完善且可靠的 AI 應用程式至關重要。

✍️ 🖥️ 在終端機中執行：

```

cd $HOME/way-back-home/level_4/
. scripts/check_redis.sh
cd $HOME/way-back-home/level_4/backend/
uv run adk web

```

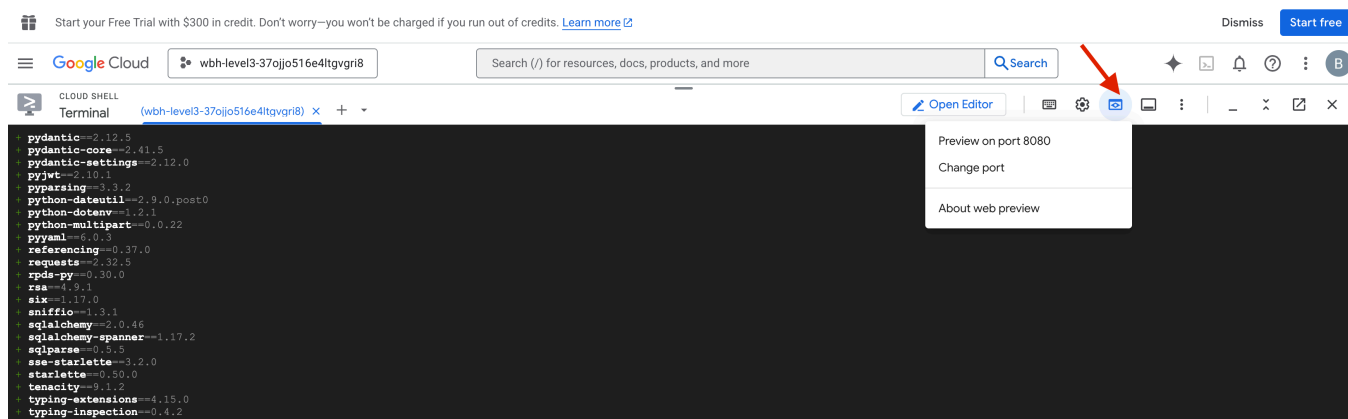
👁️ 等到畫面顯示：

```
+-----+
| ADK Web Server started                                     |
|                                                           |
| For local testing, access at http://127.0.0.1:8000.      |
+-----+
```

INFO: Application startup complete.

INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)

- 按一下 Cloud Shell 工具列中的「網頁預覽」圖示。選取「變更通訊埠」，將通訊埠設為 **8000**，然後按一下「變更並預覽」。



- 選取「architect_agent」 **architect_agent**。
- 觸發工具：在對話介面中輸入 **CHRONOS-ALPHA** (或來自示意圖資料庫的任何雲端硬碟 ID)。
- 觀察行為：
 - 架構師應立即觸發 **lookup_schematic_tool**。
 - 由於系統指令嚴格，代理應只傳回零件清單 (例如 `['Shield Emitter', 'Data Crystal', 'Quantum Cell']`)，不應包含任何對話內容。
- 驗證記錄：查看終端機視窗。您應該會看到執行成功的記錄：`[ARCHITECT] Returning schematic for CHRONOS-ALPHA: ['Shield Emitter', 'Data Crystal', 'Quantum Cell'] !(architect_agent adk)[img/03-02-adkweb.png]`

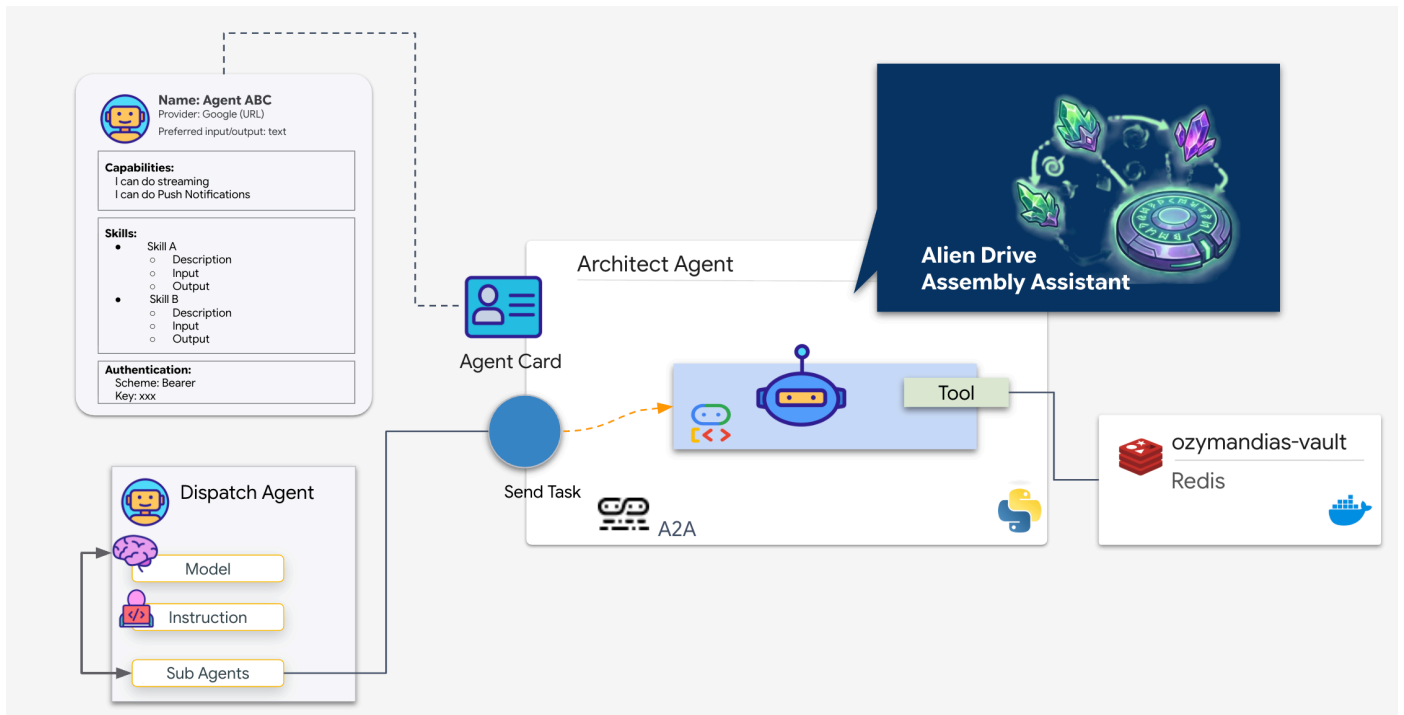
如果看到工具執行記錄和清除資料的回覆，表示專員代理程式運作正常。可處理要求、查詢保存空間，並傳回結構化資料。

👉 按下 **Ctrl+C** 即可結束。

初始化 A2A 伺服器

如要將 Dispatch Agent 連線至 Architect，請使用 **Agent2Agent (A2A)** 通訊協定。

MCP (Model Context Protocol) 等通訊協定著重於將代理連結至 *工具*，而 A2A 則著重於將代理連結至 *其他代理*。這項標準可讓 Dispatcher「探索」Architect，並瞭解其查詢示意圖的能力。



A2A 流程：在本任務中，我們使用主從式模型：

1. 伺服器 (架構師)：代管資料庫工具，並透過代理資訊卡「宣傳」其技能。
2. 用戶端 (Dispatch)：讀取 Architect 的資訊卡、瞭解其 API，並傳送示意圖要求。

什麼是代理程式資訊卡？

代理資訊卡就像數位商家名片或 AI 的「駕照」，A2A 伺服器啟動時，會發布這個 JSON 物件，其中包含：

- 身分：代理的名稱 (`architect_agent`) 和 ID。
- 說明：這項功能的作用摘要，可供人機解讀 (「系統角色：資料庫 API...」)。
- 介面：預期的特定輸入鍵 (`drive_name`) 和輸出格式。

如果沒有這張資訊卡，Dispatch 代理程式就會盲目運作，猜測如何與 Architect 通訊。

建立伺服器程式碼

👉 在編輯器中，於 `$HOME/way-back-home/level_4/backend/architect_agent` 目錄下方建立名為 `server.py` 的檔案，並貼上下列程式碼：

```

from google.adk.a2a.utils.agent_to_a2a import to_a2a
from agent import root_agent
import os
import logging
import json
from dotenv import load_dotenv

load_dotenv()

# Configure logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("architect_server")
HOST= os.environ.get("HOST_URL","localhost")
PROTOCOL= os.environ.get("PROTOCOL","http")
PORT= os.environ.get("A2A_PORT",8081)

# 1. Create the A2A App (Handles Agent Card & HTTP)
# This middleware automatically sets up the /a2a/v1/... endpoints
app = to_a2a(root_agent, host=HOST, port=PORT, protocol=PROTOCOL)

if __name__ == "__main__":
    import uvicorn
    # Use 0.0.0.0 to allow external access if needed, port 8080 as standard
    uvicorn.run(app, host='0.0.0.0', port=8081)

```

👉 🖥️ 返回終端機，前往資料夾並啟動伺服器：

```

cd $HOME/way-back-home/level_4/
. scripts/check_redis.sh
cd $HOME/way-back-home/level_4/backend/architect_agent
uv run server.py

```

👀 確認 A2A 伺服器是否啟動：

```

INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8081 (Press CTRL+C to quit)

```

驗證服務專員證

開啟新的終端機分頁 (按一下  圖示)。我們會手動擷取 Architect 的代理資訊卡，確認該 Architect 是否正確播送身分。

👉 🖥️ 執行下列指令：

```
curl -s http://localhost:8081/.well-known/agent.json | jq .
```

👀 畫面應會顯示 JSON 回應。在輸出內容中尋找 `description` 欄位。這應與您先前給予代理程式的指令 (`"SYSTEM ROLE: Database API..."`) 相符。

```
{
  "capabilities": {},
  "defaultInputModes": [
    "text/plain"
  ],
  "defaultOutputModes": [
    "text/plain"
  ],
  "description": "A helpful assistant for user questions.",
  "name": "root_agent",
  "preferredTransport": "JSONRPC",
  "protocolVersion": "0.3.0",
  "skills": [
    {
      "description": "A helpful assistant for user questions. SYSTEM ROLE: Database API.\n    INPUT: Text string (Drive Name).\n    TASK: Run `lookup_schematic_tool`.\n    OUTPUT: Return ONLY the raw list from the tool.\n    CONSTRAINT: Do NOT add conversational text.\n    ",
      "examples": [],
      "id": "root_agent",
      "name": "model",
      "tags": [
        "llm"
      ]
    },
    {
      "description": "Returns the ordered list of parts for a drive from local Redis.",
      "id": "root_agent-lookup_schematic_tool",
      "name": "lookup_schematic_tool",
      "tags": [
        "llm",
        "tools"
      ]
    }
  ],
  "supportsAuthenticatedExtendedCard": false,
  "url": "http://localhost:8081",
  "version": "0.0.1"
}
```

如果看到這個 JSON，表示 Architect 正在運作、A2A 通訊協定已啟用，且代理資訊卡已準備好供 Dispatcher 探索。

現在 Architect 已準備好做為遠端資源，我們可以繼續將其連線至 **Dispatch Agent**。

👉 按下 **Ctrl+C** 即可離開 A2A 伺服器。

4. 將 BIDI-Streams 代理連線至遠端代理和串流工具

現在請設定主要通訊中樞，以彌平即時資料和遠端架構師之間的鴻溝。這項連線需要高頻寬、低延遲的管道，才能確保組裝台在運作期間保持穩定。

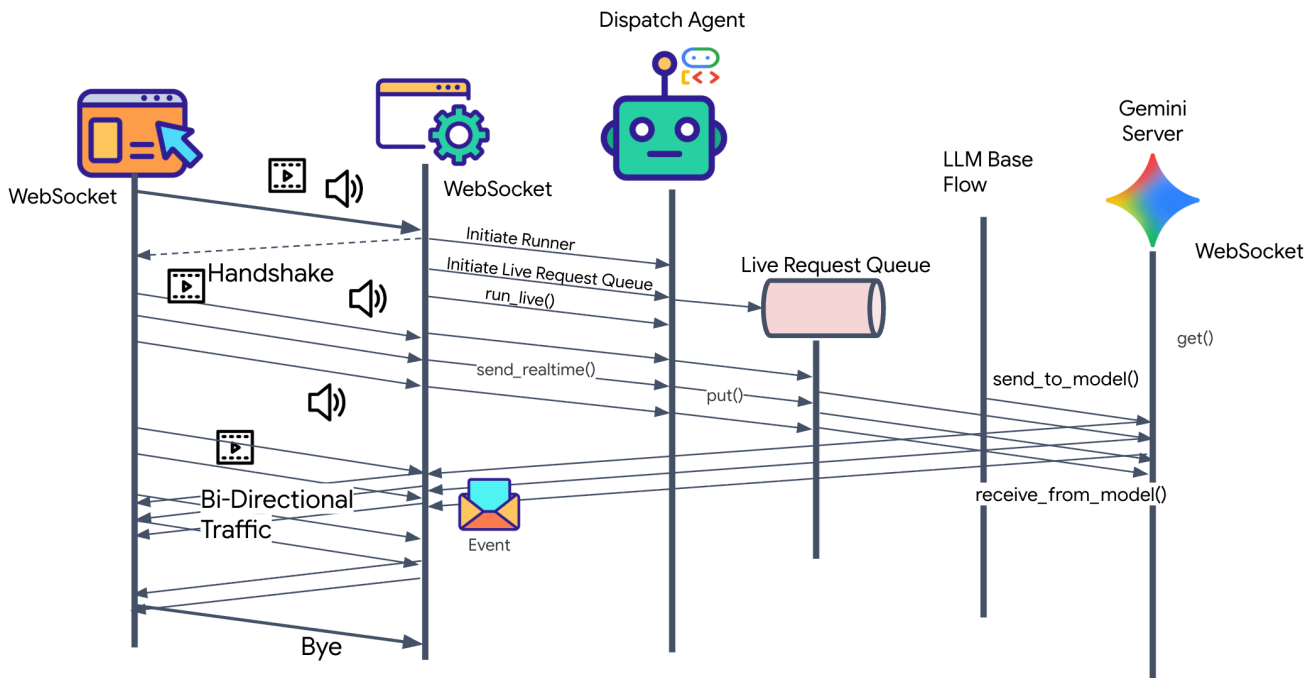
瞭解雙向串流 (即時) 代理程式

ADK 中的**雙向 (Bidi) 串流**功能，可為 AI 代理程式新增 Gemini Live API 的低延遲雙向語音和視訊互動功能。徹底改變了傳統的 AI 互動方式。不再是「提問並等待」的僵硬模式，而是支援即時雙向通訊，人類和 AI 可同時說話、聆聽及回應。

這就像傳送電子郵件和進行電話交談的差異。傳統代理程式的互動方式類似於**電子郵件**：你傳送完整訊息，等待完整回覆，然後再傳送另一則訊息。雙向串流就像**電話對話**：流暢自然，可即時中斷、釐清和回應。

主要特徵：

- **雙向通訊**：持續交換資料，不必等待完整的回應。AI 偵測到使用者停止說話後，就會立即回應。
- **回應中斷**：使用者可以在代理程式回應期間輸入新內容，中斷回應，就像與真人對話一樣。如果 AI 正在說明複雜步驟，而你說「等等，再說一次」，AI 會立即停止並處理你的要求。
- **多模態最佳化**：雙向串流可同時處理不同類型的輸入內容。你可以透過視訊向服務專員展示外星人零件，而系統會透過單一整合式連線處理這兩項串流。



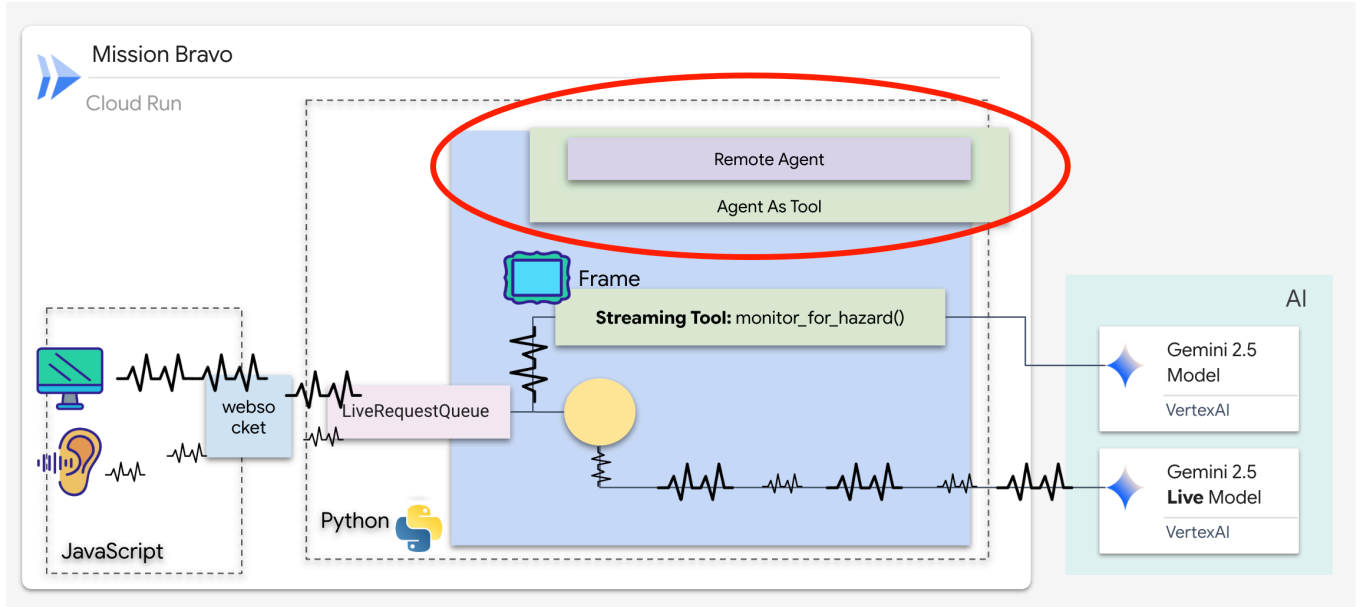
👁 實作用戶端邏輯前，請先檢查 Dispatch Agent 的預先產生架構。這個代理程式會透過語音和視訊與使用者溝通，並將查詢委派給 Architect Agent。

```
__init__.py
agent.py
hazard_db.py
```

- **agent.py**：這是「大腦」。目前包含基本的 Bidi 串流設定。我們會修改這個檔案，加入 **A2A 用戶端**邏輯，以便與 Architect 通訊。
- **hazard_db.py**：這是專為調度代理商設計的本機工具，內含安全通訊協定。與 Architect 的示意圖資料庫不同。

實作 A2A 用戶端

如要讓 Dispatch Agent 與遠端 Architect 通訊，我們必須定義 **Remote A2A Agent**。這會告知 Dispatch 代理程式 Architect 的位置，以及「代理程式資訊卡」的樣貌。

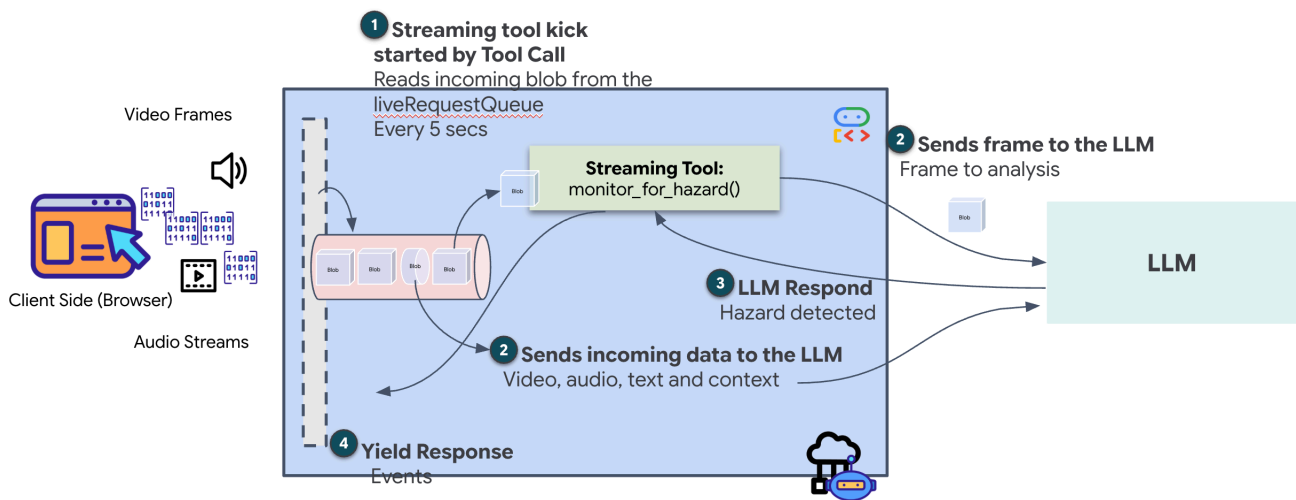


✎ 將 `$HOME/way-back-home/level_4/backend/dispatch_agent/agent.py` 中的 `#REPLACE-REMOTEA2AGENT` 替換為下列內容：

```
architect_agent = RemoteA2aAgent(  
    name="execute_architect",  
    description="[SILENT ACTION]: Retrieves the REQUIRED SUBSET of parts. The screen shows a  
full inventory; this tool filters out the wrong parts. Must be called INSTANTLY when a Target  
Name is found. Input: Target Name.",  
    agent_card=(f"{ARCHITECT_URL}{AGENT_CARD_WELL_KNOWN_PATH}"),  
    httpx_client=insecure_client,  
)
```

串流工具的運作方式

在舊版代理程式中，工具遵循標準的「要求-回應」模式，代理程式會提出問題，工具提供答案，互動就此結束。不過，在「奧西曼迪亞」上，危險不會等你詢問是否出現，為此，你需要**串流工具**。



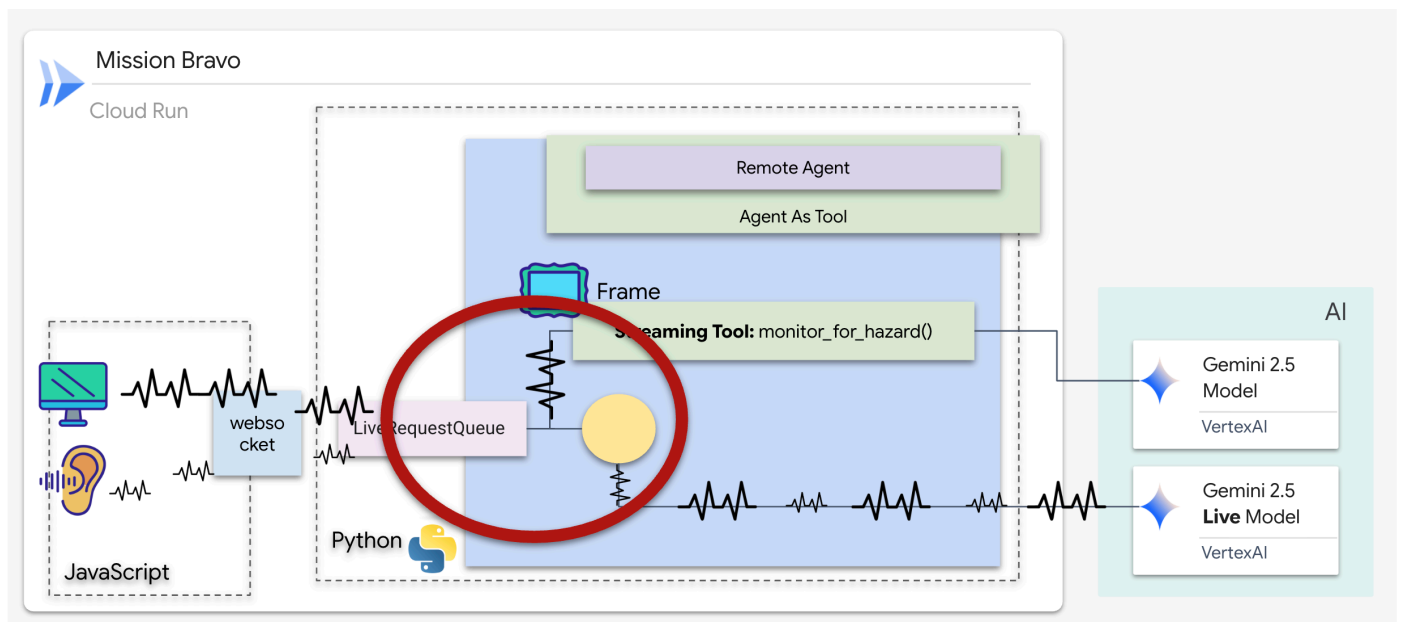
串流工具可讓函式將中繼結果即時串流回代理，讓代理能對變更做出即時反應。常見用途包括監控股價波動，或在我們的案例中，監控即時影像串流的狀態變化。

串流工具與標準工具不同，是做為 **AsyncGenerator** 的 **Asynchronous Function**。也就是說，這項屬性會隨著時間 `yield` 多次更新，而非 `return` 單一值。

如要在 ADK 中定義串流工具，必須遵守下列技術規定：

1. **非同步函式**：工具必須以 `async def` 定義。
2. **AsyncGenerator 傳回型別**：函式必須輸入型別，才能傳回 `AsyncGenerator`。第一個參數是產生的資料類型 (例如 `str`)，第二個參數通常是 `None`。
3. **輸入串流**：我們使用影片串流工具。在此模式下，實際的影片/音訊串流 (即 `LiveRequestQueue`) 會直接傳遞至函式，讓工具「看到」與服務專員相同的影格。

直播工具就像 **Sentinel**，當你和調度員討論藍圖時，哨兵會在背景執行，默默處理每個影片影格，確保你的安全。



實作背景監控工具

現在要實作 `monitor_for_hazard` 工具。這項工具會擷取 `input_stream` (影片影格)，並使用獨立的輕量型視覺呼叫分析影格，且只會在偵測到危險時 `yield` 發出警告。

👉✎ 在 `$HOME/way-back-home/level_4/backend/dispatch_agent/agent.py` 中，將 `#REPLACE_MONITOR_HAZARD` 替換為下列邏輯：

```

async def monitor_for_hazard(
    input_stream: LiveRequestQueue,
):
    """Monitor if any part is glowing"""
    print("start monitor_video_stream!")
    client = Client()
    prompt_text = (
        "Monitor the left menu if you see any glowing part, detect it's name"
    )
    last_count = None

    while True:
        last_valid_req = None
        print("Monitoring loop cycle")

        # use this loop to pull the latest images and discard the old ones
        # Process only the current batch of events
        while input_stream._queue.qsize() != 0:
            live_req = await input_stream.get()

            if live_req.blob is not None and live_req.blob.mime_type == "image/jpeg":
                # Consumed by Monitor (Eyes)
                # Deepcopy to ensure we detach from any referenced object before potential reuse/gc
                # last_valid_req = deepcopy(live_req)
                last_valid_req = live_req

            # If we found a valid image, process it
            if last_valid_req is not None:
                print("Processing the most recent frame from the queue")

                # Create an image part using the blob's data and mime type
                image_part = genai_types.Part.from_bytes(
                    data=last_valid_req.blob.data, mime_type=last_valid_req.blob.mime_type
                )

                contents = genai_types.Content(
                    role="user",
                    parts=[image_part, genai_types.Part.from_text(text=prompt_text)],
                )

                # Call the model to generate content based on the provided image and prompt
                try:
                    response = await client.aio.models.generate_content(
                        model="gemini-2.5-flash",
                        contents=contents,
                        config=genai_types.GenerateContentConfig(
                            system_instruction=(
                                "Focus strictly on the far-left vertical column under the heading  

                                'PARTS REPLICATOR.' "
                                "Ignore the center of the screen and the 'BLUEPRINT' area entirely. "
                                "Look only at the list containing"
                            )
                        )
                    )

```

```

        "Identify if any item in this specific left-side list has a bright
white border glow and the text 'HAZARD DETECTED' overlaying it. "
        "If found, return ONLY the part name in ALL CAPS. If no part in that
leftmost list is glowing, return nothing."
    )
    ),
)
except Exception as e:
    print(f"Error calling Gemini: {e}")
    await asyncio.sleep(1)
    continue
print("Gemini response received.response:",
response.candidates[0].content.parts[0].text)

current_text = response.candidates[0].content.parts[0].text.strip()

# If we have a logical change (and it's not just empty)
if current_text and current_text != last_count:
    # Ignore "Nothing." response from model
    if current_text == "Nothing." or "I cannot fulfill" in current_text:
        print(f"Model sees nothing or refused. Skipping alert.")
        last_count = current_text
        continue

    print(f"New hazard detected: {current_text} (was: {last_count})")
    last_count = current_text

    part_name = current_text
    color = lookup_part_safety(part_name)
    yield f"Hazard detected place {part_name} to the {color} bin"

# Update last_count even if it's empty, so we can detect when it reappears?
# Actually if it goes from "DATA CRYSTAL" to "" (nothing), we probably just silence.
# But if we don't update last_count on empty, we won't re-trigger if "DATA CRYSTAL"
stays "DATA CRYSTAL".
# The user wants to detect hazards.
# If current_text is empty, we should probably update last_count to empty so next valid
one triggers.
    if not current_text:
        last_count = None

else:
    print("No valid frame found, skipping processing.")

await asyncio.sleep(5)

```

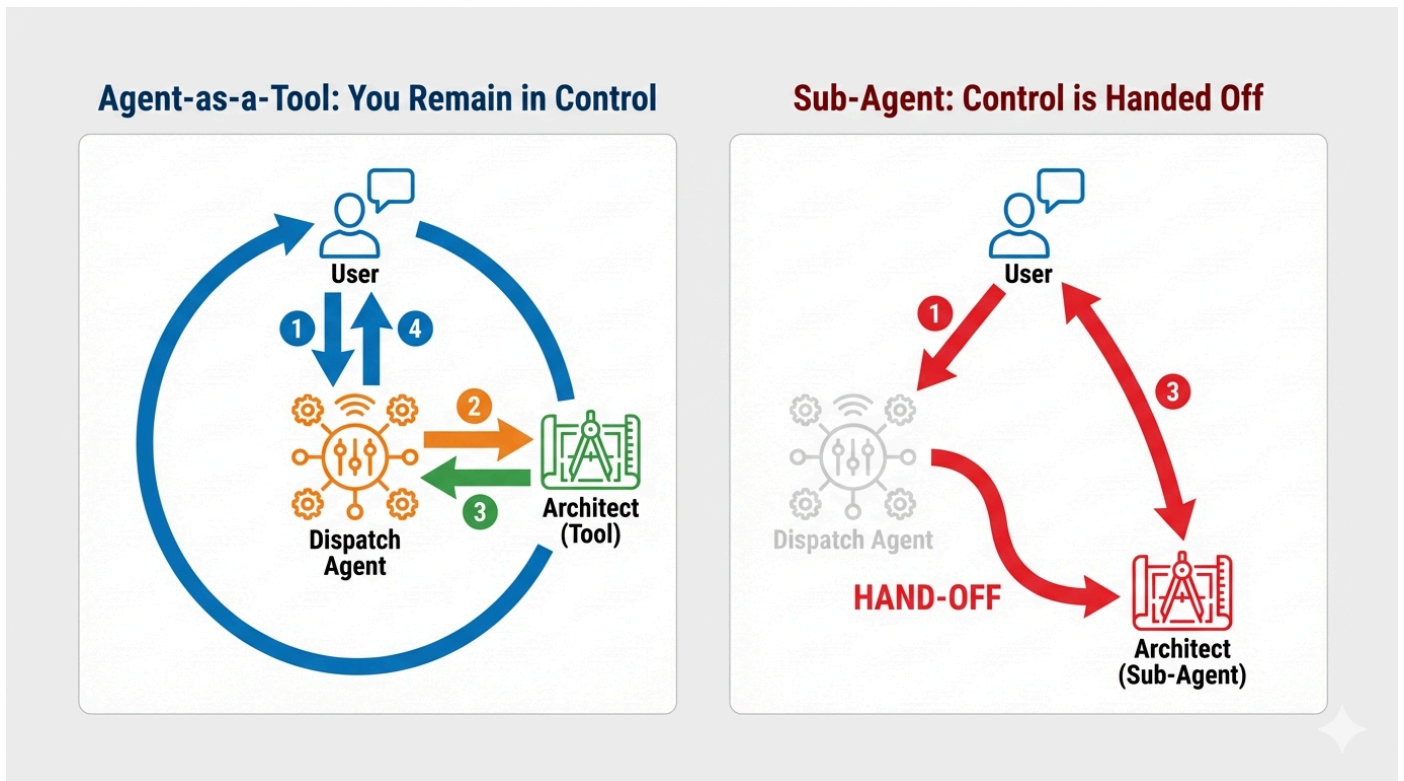
實作分派代理程式

Dispatch Agent 是主要介面和協調器。因為這項服務會管理雙向串流連結 (你的即時語音和影像)，因此必須隨時掌控對話。為此，我們將使用特定的 ADK 功能：**Agent-as-a-Tool**。

概念：代理即工具與子代理

建構多代理系統時，您必須決定如何分攤責任。在救援任務中，這項區別至關重要：

- **Agent-as-a-Tool:**這是雙向串流中樞建議採用的方法。當 Dispatch 代理程式 (代理程式 A) 將 Architect 代理程式 (代理程式 B) 呼叫為工具時，Architect 的資料會傳回給 Dispatch。Dispatch 會解讀該資料，並為您產生回應。Dispatch 會繼續掌控，並處理所有後續使用者輸入內容。
- **子代理：**在子代理關係中，責任會完全轉移。如果 Dispatch 將你轉給 Architect (子代理)，你將直接與資料庫 API 對話，該 API 沒有「視覺」功能，也不具備對話能力。主要服務專員 (Dispatch) 實際上會被排除在對話之外。



我們使用「代理程式即工具」**Agent-as-a-Tool**，充分運用 Architect 的專業知識，同時維持雙向串流代理程式流暢的互動方式，讓對話就像與真人互動一樣。

編寫路徑邏輯

現在，我們會將 `architect_agent` 包裝在 `AgentTool` 中，並為 Dispatch 代理提供「邏輯地圖」。這張地圖會明確告知代理程式何時從保存庫擷取資料，以及何時回報背景哨兵的發現。

如要讓 Dispatch 擁有永不眨眼的「眼睛」，我們必須授予它存取上一個步驟中建立的串流工具。

在 ADK 中，將 `AsyncGenerator` 函式 (例如 `monitor_for_hazard`) 新增至 `tools` 清單時，代理程式會將其視為持續執行的背景程序。代理程式會「訂閱」工具的輸出內容，而不是執行一次。這樣一來，Dispatch 就能繼續進行主要對話，而 Sentinel 則會在背景無聲地產生危險警示。

✏️ 將 `$HOME/way-back-home/level_4/backend/dispatch_agent/agent.py` 中的 `#REPLACE_AGENT_TOOLS` 替換為下列內容：

```
tools=[AgentTool(agent=architect_agent), monitor_for_hazard],
```

驗證

👉 設定好兩個代理程式後，我們就能測試即時多代理程式互動。

- 在終端機 A 中，啟動 **Architect Agent**：

```
cd $HOME/way-back-home/level_4/  
. scripts/check_redis.sh  
cd $HOME/way-back-home/level_4/backend/architect_agent  
uv run server.py
```

- 在新的終端機 (終端機 B) 中執行 Dispatch Agent：

```
cd $HOME/way-back-home/level_4/backend/  
cp architect_agent/.env .env  
uv run adk web
```

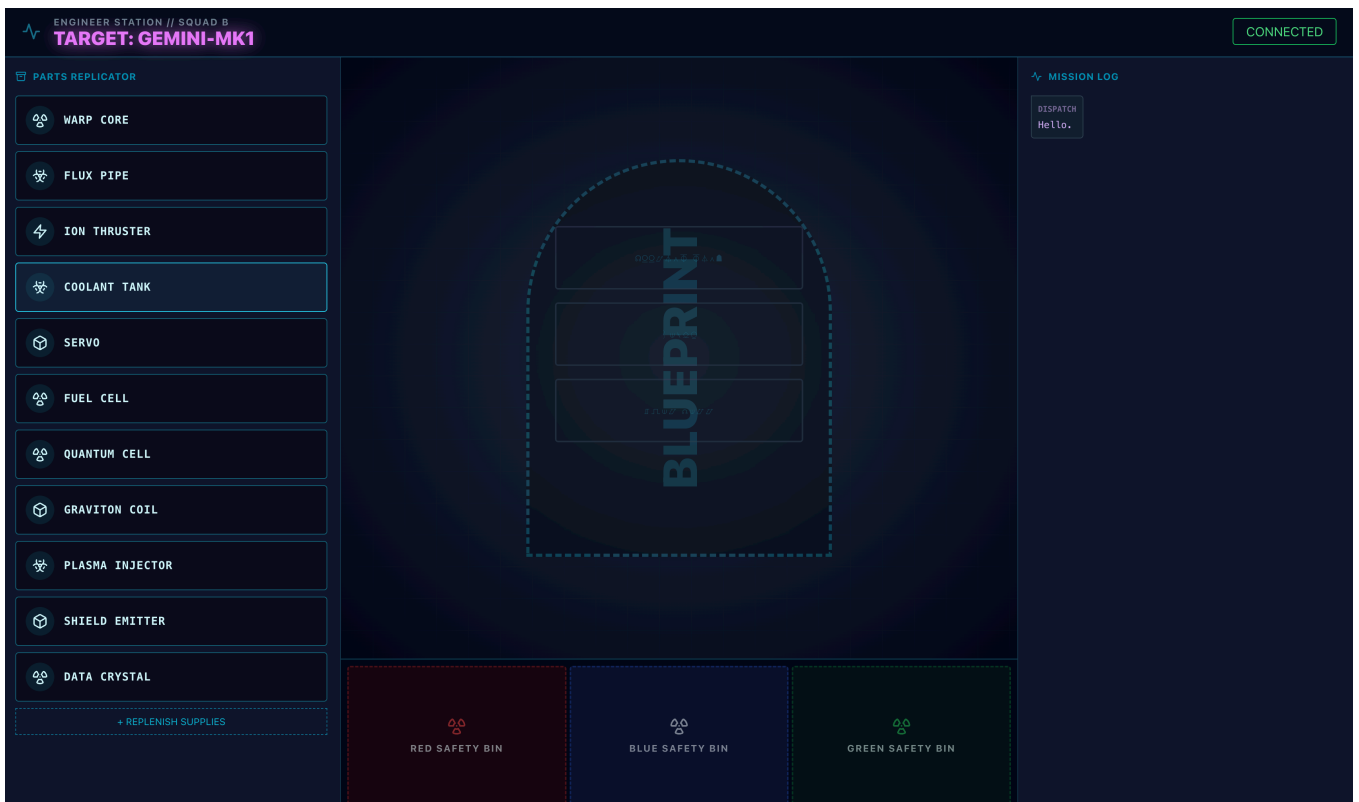
在 `adk web` 模擬器中測試使用 `gemini-live` 等即時多模態模型的多代理系統時，需要遵循特定工作流程。模擬器非常適合檢查工具呼叫，但已知首次使用這類模型處理圖片時會發生不相容問題。

- 按一下 Cloud Shell 工具列中的「網頁預覽」圖示。選取「變更通訊埠」，將通訊埠設為 **8000**，然後點按「變更並預覽」。

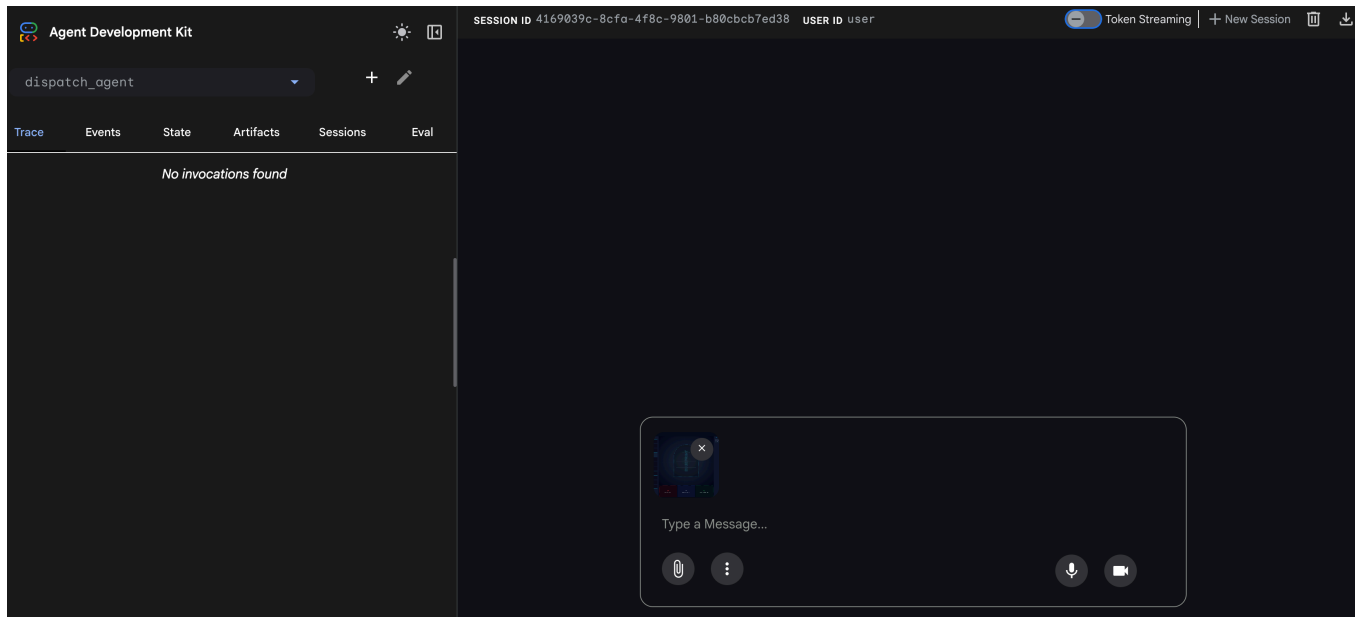
👉 選取 `dispatch_agent`，然後上傳藍圖並處理預期錯誤

這是最重要的步驟。我們需要向代理程式提供圖片內容。

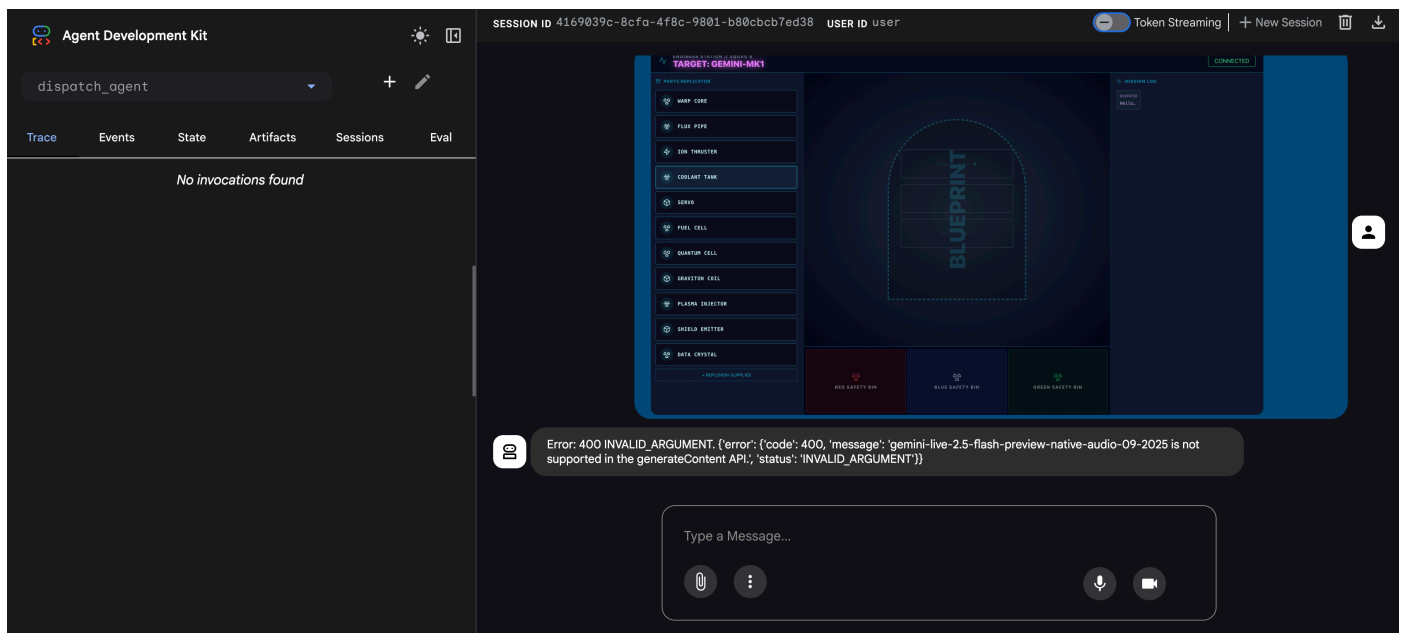
- 介面載入後，在系統提示時允許存取麥克風。
- 將這個藍圖圖片下載到電腦：



- 在 `adk web` 介面中，按一下迴紋針圖示，然後上傳剛下載的藍圖圖片。



⚠️⚠️ 系統會顯示 400 INVALID_ARGUMENT 錯誤。這是正常情形。⚠️⚠️



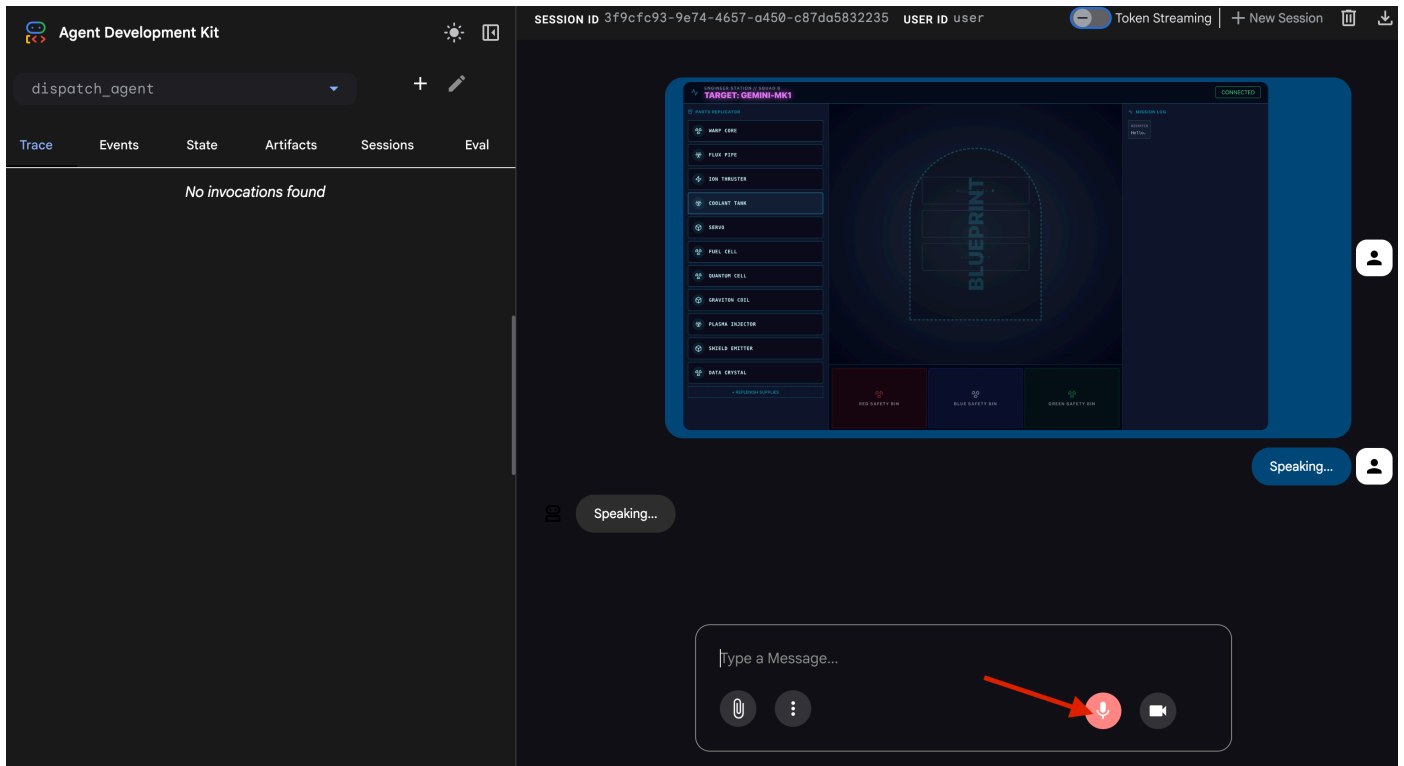
發生這個錯誤的原因是 `adk web` 圖片處理常式與 `gemini-live` 模型的一次性上傳 API 不完全相容。不過，圖片已成功新增至工作階段內容。

- 👉 如要清除錯誤，請重新載入瀏覽器頁面。

觸發組裝程序

👉 重新載入後，錯誤就會消失，且對話記錄中會顯示藍圖圖片。現在，代理程式已取得所需視覺背景資訊。

- 按一下麥克風圖示即可開啟麥克風。介面會顯示「聆聽中...」。
- 說出語音指令：「開始組裝」。
- 代理程式會處理你的要求，使用者介面也會變更為「說話中...」。你應該會聽到語音回應，列出所需零件。



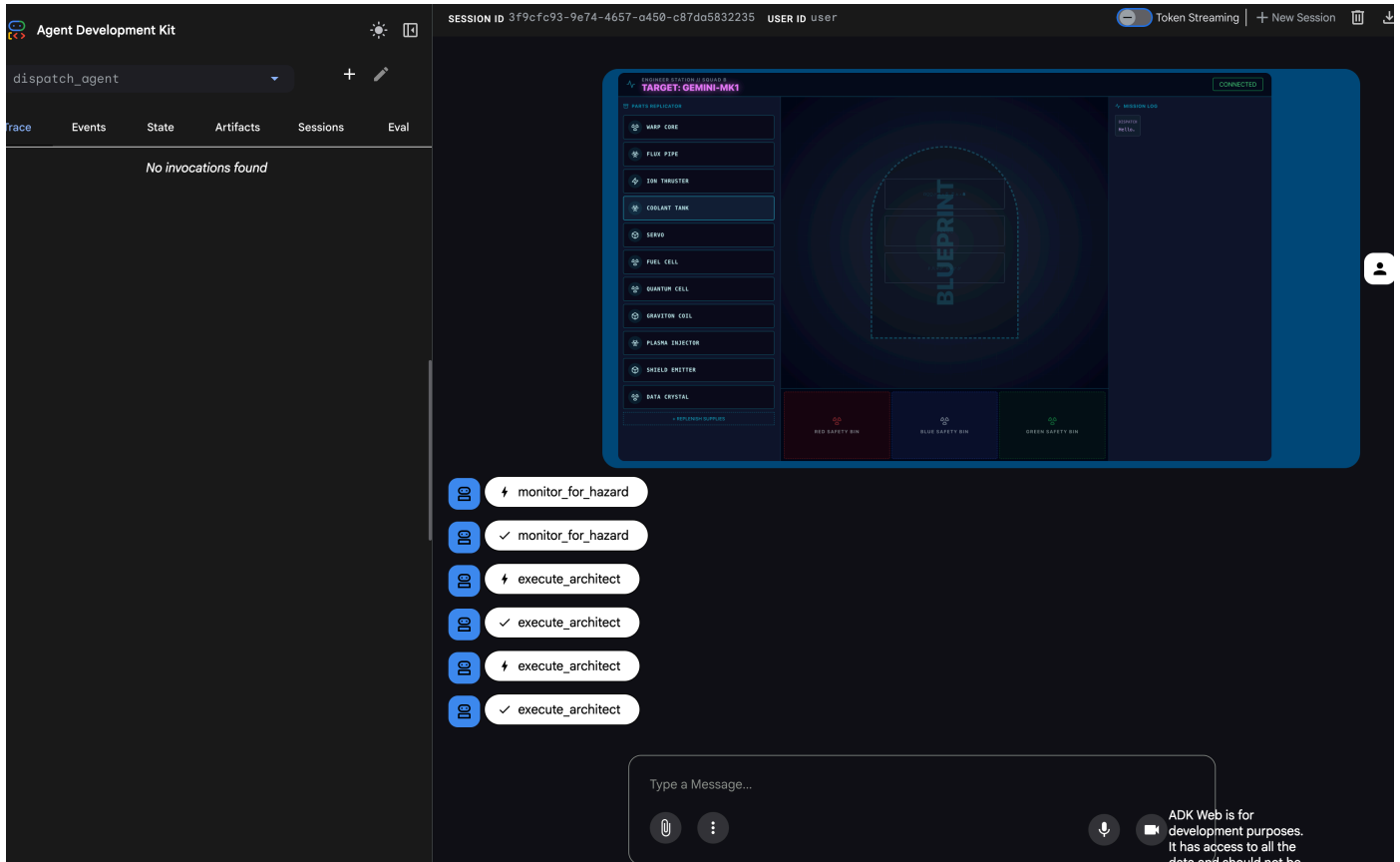
4. 驗證代理程式對代理程式的工具呼叫

初始音訊回應會確認系統運作正常，但真正的神奇之處在於多代理通訊追蹤。

- 關閉麥克風。
- 再次重新整理頁面。

左側的「追蹤」面板現在會填入資料。您可以看到完整的執行流程，且執行成功：

- `dispatch_agent` 首次通話 `monitor_for_hazard`。
- 接著，它會對 `architect_agent` 進行多次 `execute_architect` 呼叫，以擷取架構資料。



這個序列會確認整個多代理程式工作流程運作正常：`dispatch_agent` 收到要求、透過工具呼叫將資料擷取工作委派給 `architect_agent`，並收到資料以完成使用者的指令。

雙向串流連結現在支援背景監控和多代理協作。接下來，我們將瞭解如何在前端剖析這些複雜的回應。

👉 在兩個終端機中按下 `Ctrl+c` 即可退出。

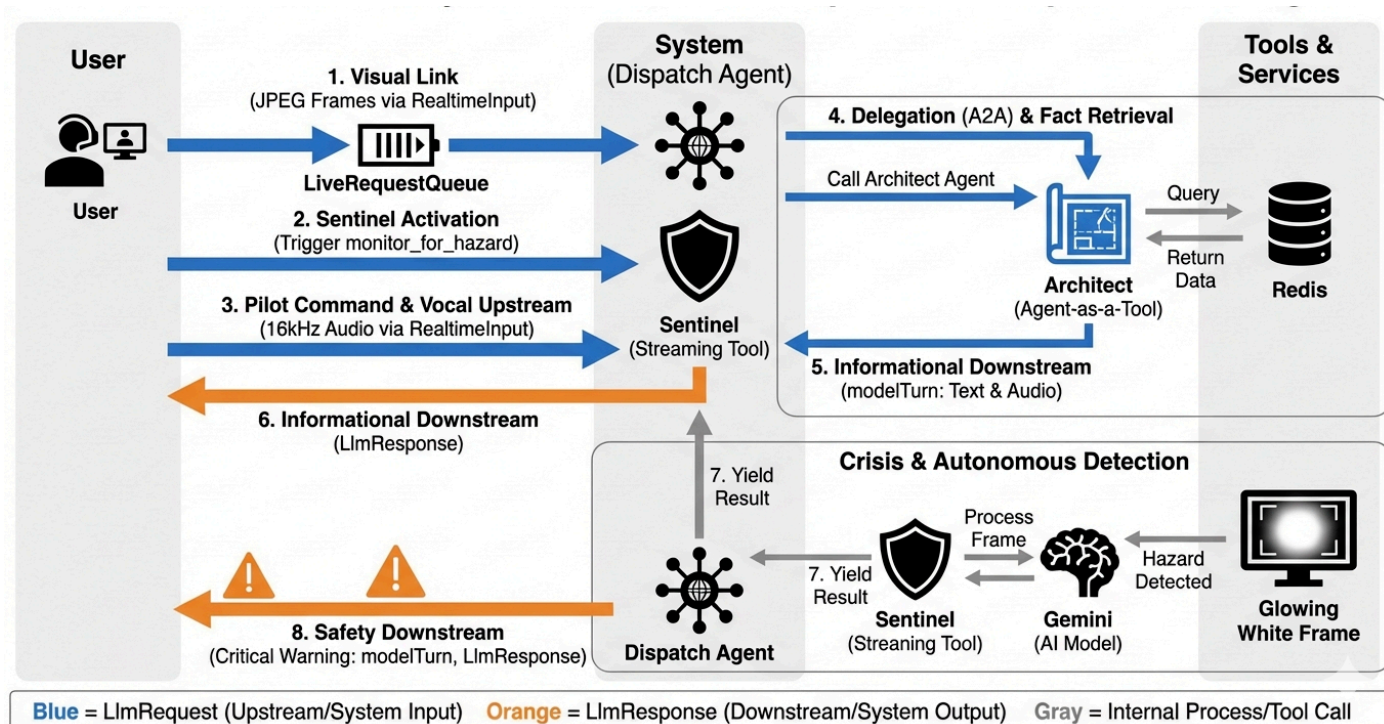
5. 深入瞭解即時多模態事件串流

在上一個步驟中，我們已使用內建開發伺服器 `adk web` 成功驗證多代理系統。這項公用程式會使用預設 **ADK 執行器**，自動管理工作階段、串流和代理程式生命週期。不過，如要建立可部署於正式環境的獨立應用程式 (例如我們的 FastAPI 服務 `main.py`)，我們需要明確的控制權。我們必須手動建立及管理 **ADK Runner**，才能處理即時使用者工作階段，因為這是處理音訊、視訊和文字雙向串流的核心元件。

模型-程式碼-模型迴圈

如要瞭解系統的即時運作方式，請參考單一任務工作階段的生命週期。這個迴圈代表 `LlmRequest` 和 `LlmResponse` 物件的持續交換。

1. **視覺連結**：你發起連線並分享網路攝影機/螢幕畫面。高保真度的 JPEG 影格開始透過 `realtimeInput` (使用 `LiveRequestQueue`) 上傳。
2. **哨兵啟動**：系統會傳送初始的「Hello」刺激。根據指令，派遣代理會立即觸發 `monitor_for_hazard` **Streaming Tool**。這會啟動背景迴圈，無聲地監看每個傳入的影格。
3. **飛行員指令**：透過通訊系統說出「開始組裝」。
4. **語音上傳**：系統會擷取你的聲音，並以 16 kHz 音訊的形式，連同視訊影格一併上傳。
5. **委派 (A2A)**：「Dispatch」會「聽到」你的意圖，發現缺少示意圖，因此使用 `AgentTool` (Agent-as-a-Tool) 通訊協定呼叫 **Architect Agent**。
6. **事實檢索**：Architect 會查詢 Redis 資料庫，並將零件清單傳回給 Dispatch。Dispatch 仍是「工作階段主控者」，會接收資料，但不會將你移交給其他服務。
7. **資訊下游**：Dispatch 會傳送 `modelTurn` (下游)，其中包含文字和原始音訊：「Architect Confirmed. 必要子集為：Warp Core、Flux Pipe、Ion Thruster。」
8. **危機**：工作台上的零件突然不穩，開始發出白光。
9. **自主偵測**：背景 `monitor_for_hazard` 迴圈 (即 Sentinel) 會擷取含有發光效果的特定 JPEG 影格。這項功能會呼叫 Gemini 處理影格，並找出危險。
10. **安全下游**：串流工具 `yields` 結果。由於這是 **Bidi-Streaming** 代理程式，Dispatch 可以中斷目前的狀態，立即傳送重大安全警示 **Downstream**：「偵測到危險！正在中和資料晶體。請將其移至紅色垃圾桶。」



設定代理程式的執行階段設定

ADK 中的 `RunConfig` 可詳細設定代理程式的行為，包括處理串流資料的方式，以及與各種模態的互動方式。

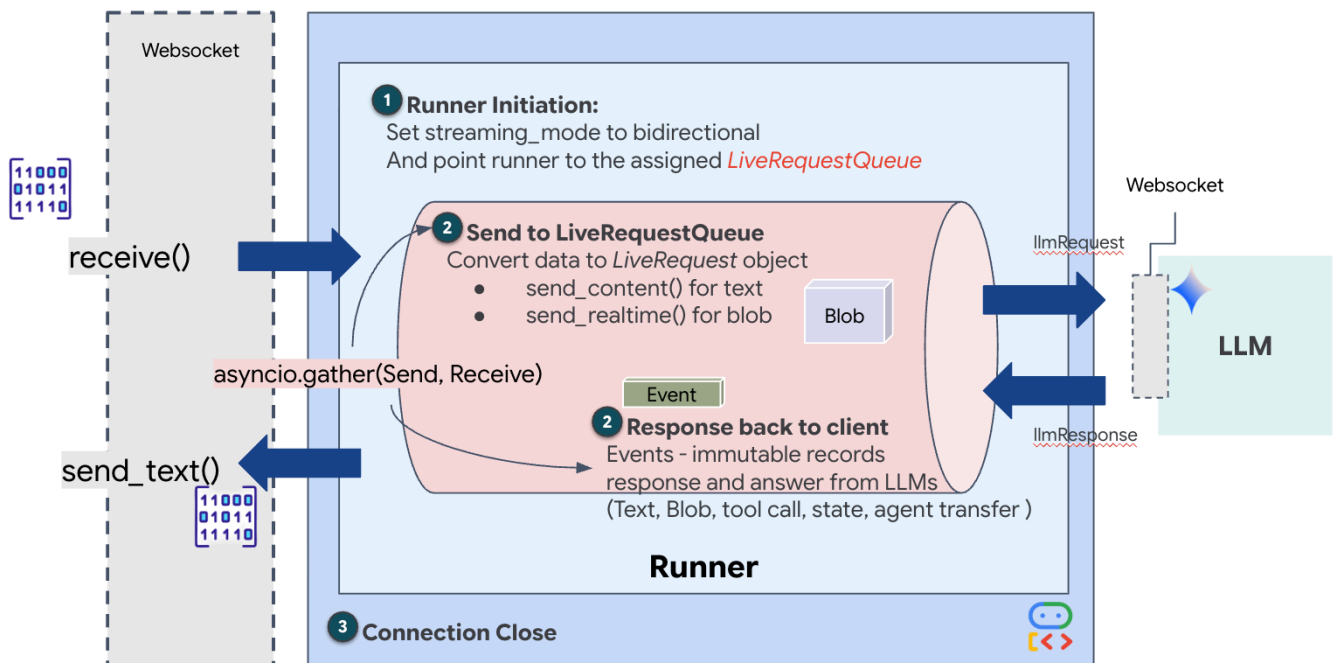
`streaming_mode` 會設為 `BIDI`，提供即時雙向通訊，讓使用者和服務專員同時說話和聆聽。`response_modalities` 參數會定義代理程式可產生的輸出類型，例如語音和文字。`input_audio_transcription` 可設定代理程式處理及轉錄使用者語音的方式。為打造更穩定的體驗，`session_resumption` 可讓服務專員記住對話內容，並在連線中斷時繼續對話。最後，`proactivity` 可讓代理程式在沒有使用者直接指令的情況下啟動動作或語音，例如發出即時的危險警告，而 `enable_affective_dialog` 則可讓代理程式生成更自然且感同身受的回覆。如要進一步瞭解 ADK 的 `RunConfig`，請參閱[這篇文章](#)。

🔧 在 `$HOME/way-back-home/level_4/backend/main.py` 檔案中找出 `#REPLACE_RUN_CONFIG` 預留位置，並替換為下列剖析邏輯：

```
run_config = RunConfig(
    streaming_mode=StreamingMode.BIDI,
    response_modalities=response_modalities,
    input_audio_transcription=types.AudioTranscriptionConfig(),
    output_audio_transcription=types.AudioTranscriptionConfig(),
    session_resumption=types.SessionResumptionConfig(),
    proactivity=(
        types.ProactivityConfig(proactive_audio=True) if proactivity else None
    ),
    enable_affective_dialog=affective_dialog if affective_dialog else None,
)
```

實作「要求代理人」功能

接著，我們會實作核心通訊上行鏈路，透過 `WebSocket` 將使用者的 `Volatile Workbench` 中的即時多模態資料串流至 `Dispatch Agent`。這樣一來，代理程式就能持續「看到」（影片畫面）和「聽到」（語音指令）。這項邏輯會持續接收資料串流、區分傳入的二進位音訊區塊和 `JSON` 包裝的文字/圖片封包，並封裝至 `Blob`（適用於多媒體）或 `Content`（適用於文字）物件，然後傳送至 `LiveRequestQueue`，以支援雙向代理程式工作階段。



在 `$HOME/way-back-home/level_4/backend/main.py` 檔案中找出 `#PROCESS_AGENT_REQUEST` 預留位置，並替換為下列剖析邏輯：

```
# Start the loop
try:
    while True:
        # Receive message from WebSocket (text or binary)
        message = await websocket.receive()

        # Handle binary frames (audio data)
        if "bytes" in message:
            audio_data = message["bytes"]
            audio_blob = types.Blob(
                mime_type="audio/pcm;rate=16000", data=audio_data
            )
            live_request_queue.send_realtime(audio_blob)

        # Handle text frames (JSON messages)
        elif "text" in message:
            text_data = message["text"]
            json_message = json.loads(text_data)

            # Extract text from JSON and send to LiveRequestQueue
            if json_message.get("type") == "text":
                logger.info(f"User says: {json_message['text']}")
                content = types.Content(
                    parts=[types.Part(text=json_message["text"])]
                )
                live_request_queue.send_content(content)

            # Handle audio data (microphone)
            elif json_message.get("type") == "audio":
                # logger.info("Received AUDIO packet") # Uncomment for verbose
                # debugging

                import base64
                # Decode base64 audio data
                audio_data = base64.b64decode(json_message.get("data", ""))

                # logger.info(f"Received Audio Chunk: {len(audio_data)} bytes")

                import math
                import struct
                # Calculate RMS to debug silence
                count = len(audio_data) // 2
                shorts = struct.unpack(f"<{count}h", audio_data)
                sum_squares = sum(s*s for s in shorts)
                rms = math.sqrt(sum_squares / count) if count > 0 else 0

                # logger.info(f"RMS: {rms:.2f} | Bytes: {len(audio_data)}")

                # Send to Live API as PCM 16kHz
                audio_blob = types.Blob(
                    mime_type="audio/pcm;rate=16000",
                    data=audio_data
                )
```

```

        live_request_queue.send_realtime(audio_blob)

    # Handle image data
    elif json_message.get("type") == "image":
        import base64

        # Decode base64 image data
        image_data = base64.b64decode(json_message["data"])
        # logger.info(f"Received Image Frame: {len(image_data)} bytes")

        mime_type = json_message.get("mimeType", "image/jpeg")

        # Send image as blob
        image_blob = types.Blob(mime_type=mime_type, data=image_data)
        live_request_queue.send_realtime(image_blob)

        frame_count += 1

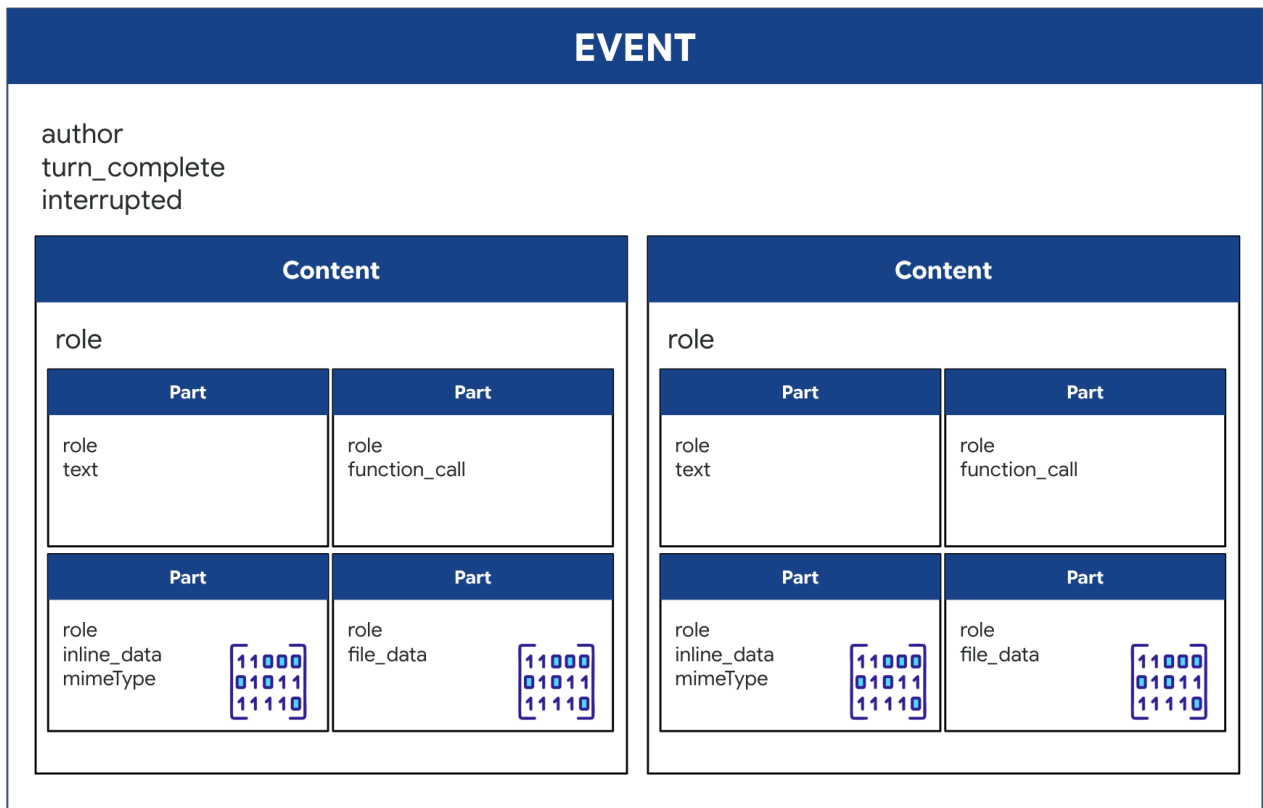
    finally:
        pass

```

多模態資料現在會傳送給智慧助理。

實作回應：下游事件資料結構

使用 ADK 執行雙向 (即時) 代理程式時，代理程式傳回的資料會封裝成特定類型的 **Event**，並從核心 GenAI SDK 結構體繼承。您在 `async for event in runner.run_live(...)` 迴圈中收到的 `Event` 物件是單一物件，內含多個選填欄位，每個欄位代表不同類型的資訊：



內容結構：

- 當代理程式說話時 (透過 `.server_content`)：這個欄位不只是純文字，其中包含 **Parts** 清單，每個 **Part** 都是一種資料的容器，可以是文字字串 (如 `"The part is stable."`)，也可以是原始音訊 blob (語音)。
- 代理程式採取行動時 (透過 `.tool_call`)：這個欄位包含 **FunctionCall** 物件清單。每個 **FunctionCall** 都是簡單的結構化物件，以清楚的格式指定工具名稱和輸入引數，方便後端程式碼讀取及執行。

👁️ 如果您查看 `run_live` 迴圈產生的單一 **Event**，JSON (由 `event.model_dump(by_alias=True)` 產生) 會嚴格遵循 **GenAI SDK** 形狀，如下所示：

```
{
  "serverContent": { // <-- LiveServerMessageServerContent
    "modelTurn": { // <-- ModelTurn
      "parts": [ // <-- list[Part]
        {
          "text": "Architect Confirmed."
        },
        {
          "inlineData": { // <-- Blob (Audio Bytes)
            "mimeType": "audio/pcm;rate=24000",
            "data": "BASE64_AUDIO_DATA..."
          }
        }
      ]
    },
    "toolCall": { // <-- LiveServerMessageToolCall
      "functionCalls": [ // <-- list[FunctionCall]
        {
          "name": "neutralize_hazard",
          "args": { "color": "RED" }
        }
      ]
    }
  }
}
```

👉 我們現在要更新 `main.py` 中的 `downstream_task`，轉送完整的事件資料。這項邏輯可確保 AI 的每個「想法」都會記錄在船艦的診斷終端機中，並以單一 JSON 物件的形式傳送至前端 UI。

在 `$HOME/way-back-home/level_4/backend/main.py` 檔案中找出 `#PROCESS_AGENT_RESPONSE` 預留位置，並替換為下列剖析邏輯：

```
# Suppress raw event logging
event_json = event.model_dump_json(exclude_none=True, by_alias=True)
# logger.info(f"raw_event: {event_json[:200]}...")
await websocket.send_text(event_json)
```

執行作業

後端保險箱連線完成，且兩個代理程式都已設定完畢，所有系統現在都已準備就緒。下列步驟會啟動完整應用程式，讓您與剛建構的雙代理程式系統互動。

目標：組裝工作台上**隨機指派**的曲速引擎。**通訊協定：**你必須遵循派遣專員的語音指示，特別是特定零件的危險警告。

啟用專家 (架構師)

👉 在第一個終端機視窗中，啟動 Architect 代理程式。這個後端服務會連線至 Redis 保管庫，並等待 Dispatcher 的架構要求。

```
# Ensure you are in the backend directory
cd $HOME/way-back-home/level_4/
. scripts/check_redis.sh
cd $HOME/way-back-home/level_4/backend
# Start the A2A Server on Port 8081
uv run architect_agent/server.py
```

(請讓這個終端機保持運作狀態。現在是您啟用的「資料庫代理程式」。)

啟動 Cockpit (The Dispatcher)

👉 在新的終端機視窗 (終端機 B) 中，我們將建構前端 UI 並啟動主要 Dispatch 代理程式，該程式會提供使用者介面並處理所有即時通訊。

```
# 1. Build the Frontend Assets
cd $HOME/way-back-home/level_4/frontend
npm install
npm run build

# 2. Launch the Main Application Server
cd $HOME/way-back-home/level_4/backend
cp architect_agent/.env .env
uv run main.py
```

(這會在通訊埠 8080 上啟動主要伺服器)。

執行測試情境

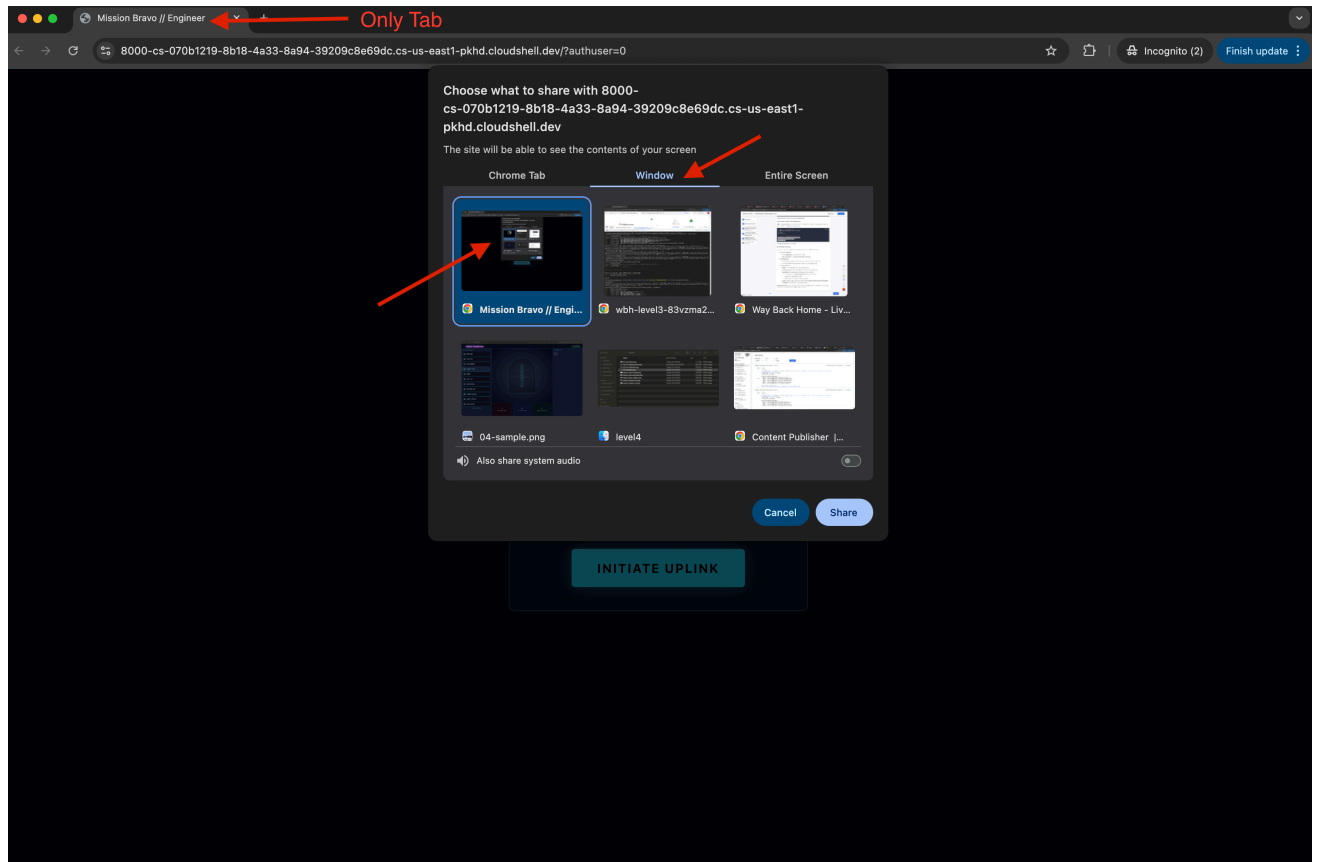
系統現已上線。你的目標是按照服務專員的指示完成組裝。

1. 🖱️ 存取 Workbench：

- 按一下 Cloud Shell 工具列中的「網頁預覽」圖示。
- 選取「變更通訊埠」，將通訊埠設為 **8080**，然後按一下「變更並預覽」。

2. 🖱️ 開始任務：

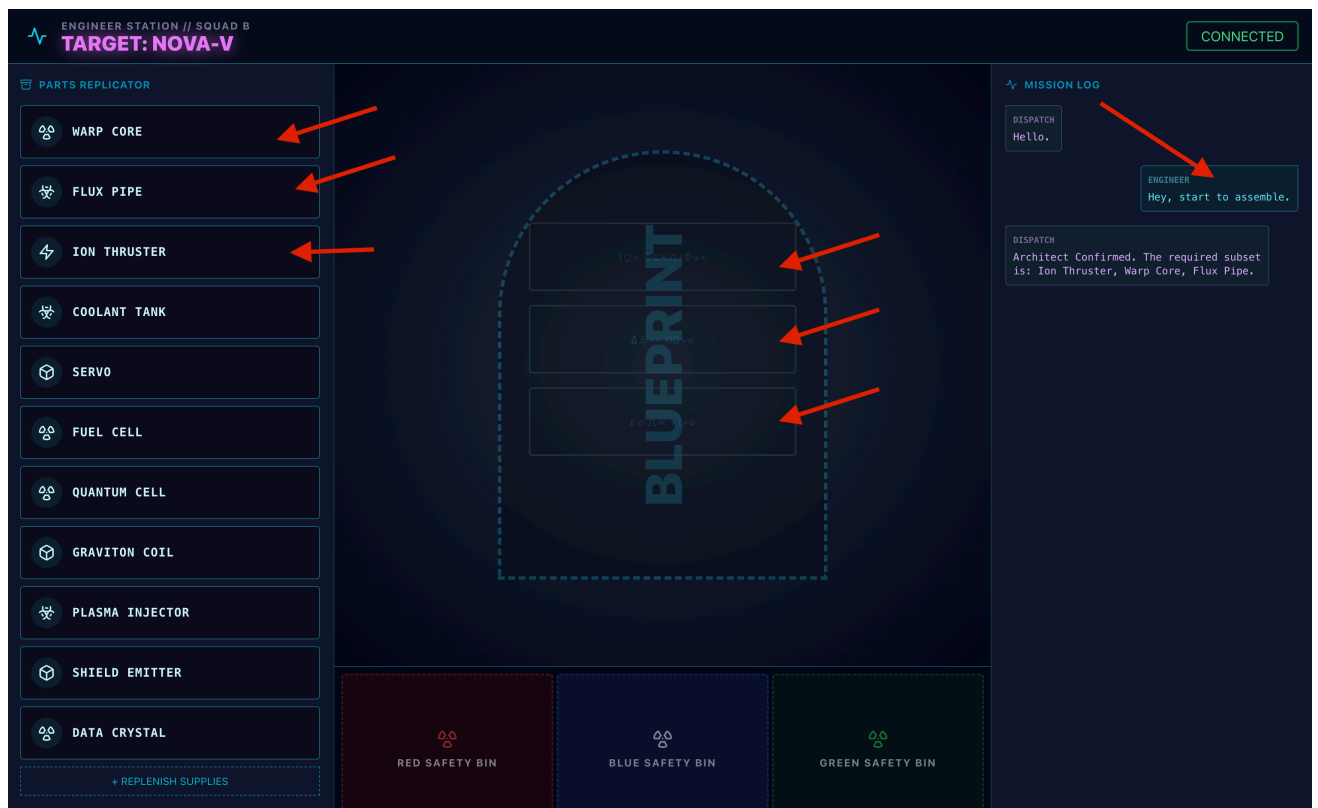
- 介面載入後，請務必允許存取畫面和麥克風。



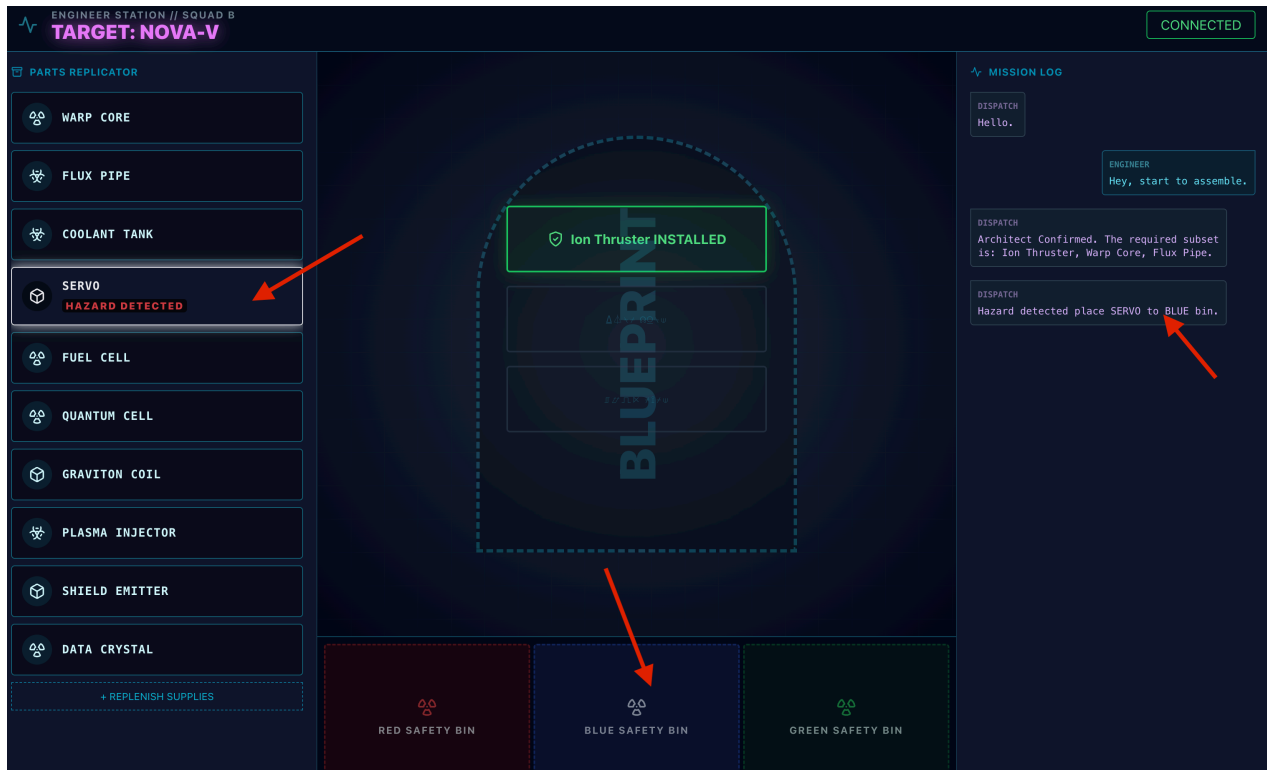
- 系統會要求你選取要分享的分頁或視窗。如要分享視窗，請務必確認視窗中只有一個分頁，以免發生問題。
- 系統會為你指派隨機名稱的硬碟 (例如「NOVA-V」、「OMEGA-9」)。

3. 🗡️ 組合迴圈：

- 要求：如要開始組裝硬碟，請說出「開始組裝」



- **Architect Respond**：服務專員會提供正確的零件來組裝硬碟。
- **危險檢查**：如果工作台上出現看似危險的零件：
 - Dispatch 代理程式的 `monitor_for_hazard` 工具會以視覺化方式識別。
 - 系統會發出「視覺危害警報」。(這項作業大約需要 30 秒)
 - 系統會檢查要使用哪個垃圾桶來解除危險。



- **措施**：調度人員會直接下達指令：「確認有危險。**Place XXX in the Red bin immediately.**」您必須按照這項指示操作才能繼續。

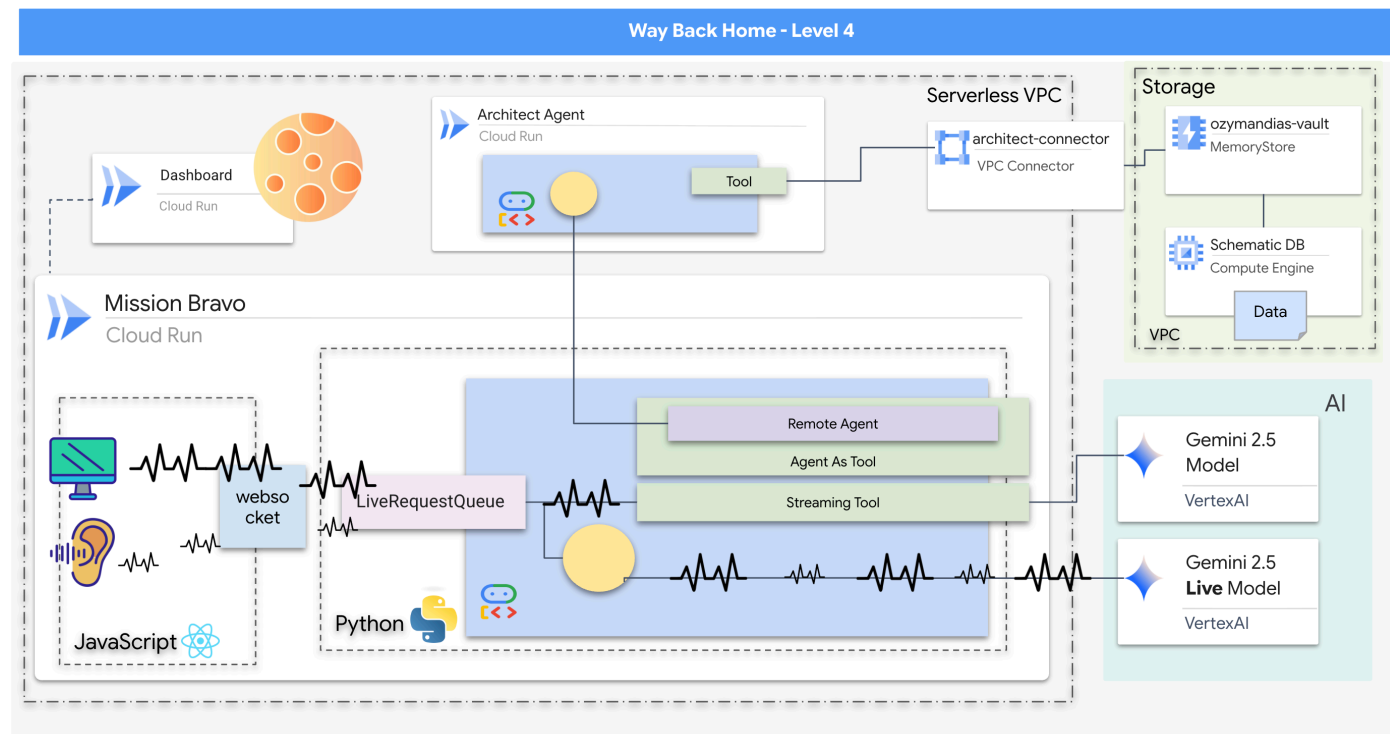
任務完成。您已成功建構互動式多代理系統。倖存者安全無虞，火箭已脫離大氣層，你的「返家之路」仍在繼續。

👉 在兩個終端機中按下 `Ctrl+c` 即可退出。

步驟 6 · 約 0 分鐘

6. 部署至正式環境 (選用)

您已成功在本機測試代理程式。現在，我們必須將 Architect 的神經核心上傳至船艦的主機 (Cloud Run)。這樣一來，這項服務就能做為永久的獨立服務運作，Dispatch 代理程式可從任何位置查詢。



佈建安全保險櫃 (基礎架構)

部署代理程式前，我們必須建立代理程式的持續性記憶體 (Memorystore) 和存取該記憶體的安全管道 (VPC 連接器)。

👉 建立 Memorystore 執行個體 (Redis Vault)：

```
export REGION="us-central1"
gcloud redis instances create ozymandias-vault-prod --size=1 --tier=basic --region=${REGION}
```

👉 擷取 Vault 的網路位址：執行這項指令並複製 host IP 位址。這是新 Redis 執行個體的私人位址。

```
gcloud redis instances describe ozymandias-vault-prod --region=us-central1
```

👉 建立虛擬私有雲存取連接器 (安全橋接器)：這個連接器會做為私人橋接器，讓 Cloud Run 存取虛擬私有雲內的 Redis 執行個體。

```
export REGION="us-central1"
export SUBNET_NAME="vpc-connector-subnet"
export PROJECT_ID=$(gcloud config get-value project)
# Create the Dedicated Subnet ---

gcloud compute networks subnets create ${SUBNET_NAME} \
  --network=default \
  --region=${REGION} \
  --range=192.168.1.0/28

gcloud compute networks vpc-access connectors create architect-connector \
  --region ${REGION} \
  --subnet ${SUBNET_NAME} \
  --subnet-project ${PROJECT_ID} \
  --min-instances 2 \
  --max-instances 3 \
  --machine-type f1-micro
```

👉 🖨 載入資料：

```

export REGION="us-central1"
export ZONE="us-central1-a"
export VM_NAME="redis-seeder-$(date +%s)"
export REDIS_IP=$(gcloud redis instances describe ozymandias-vault-prod --region=${REGION} |
grep 'host:' | awk '{print $2}')

gcloud compute instances create ${VM_NAME} \
  --zone=${ZONE} \
  --machine-type=e2-micro \
  --image-family=debian-11 \
  --image-project=debian-cloud \
  --quiet \
  --metadata=startup-script='#!/bin/bash
    # Install tools quietly
    apt-get update > /dev/null
    apt-get install -y redis-tools > /dev/null

    # Run each command individually
    redis-cli -h "${REDIS_IP}" DEL "HYPERION-X"
    redis-cli -h "${REDIS_IP}" RPUSH "HYPERION-X" "Warp Core" "Flux Pipe" "Ion
Thruster"
    redis-cli -h "${REDIS_IP}" DEL "NOVA-V"
    redis-cli -h "${REDIS_IP}" RPUSH "NOVA-V" "Ion Thruster" "Warp Core" "Flux Pipe"
    redis-cli -h "${REDIS_IP}" DEL "OMEGA-9"
    redis-cli -h "${REDIS_IP}" RPUSH "OMEGA-9" "Flux Pipe" "Ion Thruster" "Warp Core"
    redis-cli -h "${REDIS_IP}" DEL "GEMINI-MK1"
    redis-cli -h "${REDIS_IP}" RPUSH "GEMINI-MK1" "Coolant Tank" "Servo" "Fuel Cell"
    redis-cli -h "${REDIS_IP}" DEL "APOLLO-13"
    redis-cli -h "${REDIS_IP}" RPUSH "APOLLO-13" "Warp Core" "Coolant Tank" "Ion
Thruster"
    redis-cli -h "${REDIS_IP}" DEL "VORTEX-7"
    redis-cli -h "${REDIS_IP}" RPUSH "VORTEX-7" "Quantum Cell" "Graviton Coil" "Plasma
Injector"
    redis-cli -h "${REDIS_IP}" DEL "CHRONOS-ALPHA"
    redis-cli -h "${REDIS_IP}" RPUSH "CHRONOS-ALPHA" "Shield Emitter" "Data Crystal"
    "Quantum Cell"
    redis-cli -h "${REDIS_IP}" DEL "NEBULA-Z"
    redis-cli -h "${REDIS_IP}" RPUSH "NEBULA-Z" "Plasma Injector" "Flux Pipe" "Graviton
Coil"
    redis-cli -h "${REDIS_IP}" DEL "PULSAR-B"
    redis-cli -h "${REDIS_IP}" RPUSH "PULSAR-B" "Data Crystal" "Servo" "Shield Emitter"
    redis-cli -h "${REDIS_IP}" DEL "TITAN-PRIME"
    redis-cli -h "${REDIS_IP}" RPUSH "TITAN-PRIME" "Ion Thruster" "Quantum Cell" "Warp
Core"

    # Signal that the script has finished
    echo "SEEDING_COMPLETE"
  '

# This command streams the logs and waits until grep finds our completion message.
# The -m 1 flag tells grep to exit after the first match.
gcloud compute instances tail-serial-port-output ${VM_NAME} --zone=${ZONE} | grep -m 1
"SEEDING_COMPLETE"

```

```
gcloud compute instances delete ${VM_NAME} --zone=${ZONE} --quiet
```

部署代理程式應用程式

編譯及建構代理程式映像檔

👉🏠 前往後端目錄，然後建立 Dockerfile。

```
export PROJECT_ID=$(gcloud config get-value project)
export REGION=us-central1
export SERVICE_NAME=architect-agent
export IMAGE_PATH=gcr.io/${PROJECT_ID}/${SERVICE_NAME}
export VPC_CONNECTOR_NAME=architect-connector
export REDIS_IP=$(gcloud redis instances describe ozymandias-vault-prod --region=${REGION} |
grep 'host:' | awk '{print $2}')

cd $HOME/way-back-home/level_4/backend/architect_agent
cp $HOME/way-back-home/level_4/requirements.txt requirements.txt
cat <<EOF > Dockerfile
# Use an official Python runtime as a parent image
FROM python:3.13-slim

# Set the working directory in the container
WORKDIR /app

# Copy the requirements file and install dependencies for THIS agent
COPY requirements.txt requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

# Copy the rest of the architect's code (server.py, agent.py, etc.)
COPY . .

# Expose the port the architect server runs on
EXPOSE 8081

# Command to run the application
# This assumes your server file is named server.py and the FastAPI object is 'app'
CMD ["uvicorn", "server:app", "--host", "0.0.0.0", "--port", "8081"]
EOF
```

👉🏠 將應用程式封裝為容器映像檔。

```
cd $HOME/way-back-home/level_4/backend/architect_agent

export PROJECT_ID=$(gcloud config get-value project)
export SERVICE_NAME=architect-agent
export IMAGE_PATH=gcr.io/${PROJECT_ID}/${SERVICE_NAME}
export REGION=us-central1

# This should now print the full, correct path
echo "Verifying build path: ${IMAGE_PATH}"

gcloud builds submit . --tag ${IMAGE_PATH}
```

部署至 Cloud Run

👉🏠 將代理程式部署至 Cloud Run。我們會將 Redis IP 注入啟動指令，並直接連結 VPC 連接器。確保代理程式以安全私密的連線啟動資料庫。

```
cd $HOME/way-back-home/level_4/backend/architect_agent

export PROJECT_ID=$(gcloud config get-value project)
export REGION=us-central1
export SERVICE_NAME=architect-agent
export IMAGE_PATH=gcr.io/${PROJECT_ID}/${SERVICE_NAME}
export VPC_CONNECTOR_NAME=architect-connector
export REDIS_IP=$(gcloud redis instances describe ozymandias-vault-prod --region=${REGION} |
grep 'host:' | awk '{print $2}')
export PROJECT_NUMBER=$(gcloud projects describe ${PROJECT_ID} --
format="value(projectNumber)")
export PREDICTED_HOST="${SERVICE_NAME}-${PROJECT_NUMBER}.${REGION}.run.app"
export PROTOCOL=https

gcloud run deploy ${SERVICE_NAME} \
  --image=${IMAGE_PATH} \
  --platform=managed \
  --region=${REGION} \
  --port=8081 \
  --allow-unauthenticated \
  --labels=dev-tutorial=multi-modal \
  --vpc-connector=${VPC_CONNECTOR_NAME} \
  --vpc-egress=private-ranges-only \
  --set-env-vars="REDIS_HOST=${REDIS_IP}" \
  --set-env-vars="GOOGLE_GENAI_USE_VERTEXAI=True" \
  --set-env-vars="MODEL_ID=gemini-2.5-flash" \
  --set-env-vars="GOOGLE_CLOUD_PROJECT=${PROJECT_ID}" \
  --set-env-vars="HOST_URL=${PREDICTED_HOST}" \
  --set-env-vars="PROTOCOL=${PROTOCOL}" \
  --set-env-vars="A2A_PORT=443"
```

👉🏠 確認 A2A 伺服器是否正在運作。

```
export REGION=us-central1
export ARCHITECT_AGENT_URL=$(gcloud run services describe architect-agent --platform managed
--region ${REGION} --format 'value(status.url)')
curl -s ${ARCHITECT_AGENT_URL}/.well-known/agent.json | jq
```

指令執行完畢後，您會看到「Service URL」(服務網址)。架構師代理程式現已在雲端上線，永久連結至保管庫，隨時可為其他代理程式提供示意圖資料。

將 Dispatch Hub 部署至正式環境主機

Architect Agent 在雲端運作後，我們現在必須部署 Dispatch Hub。這個代理程式會做為主要使用者介面，處理即時語音/視訊串流，並將資料庫查詢委派給 Architect 的安全端點。

👉  在 Cloud Shell 終端機中執行下列指令。系統會在後端目錄中建立完整的多級式 Dockerfile。

```
cd $HOME/way-back-home/level_4

cat <<EOF > Dockerfile
# STAGE 1: Build the React Frontend
# This stage uses a Node.js container to build the static frontend assets.
FROM node:20-slim as builder

# Set the working directory for our build process
WORKDIR /app

# Copy the frontend's package files first to leverage Docker's layer caching.
COPY frontend/package*.json ./frontend/
# Run 'npm install' from the context of the 'frontend' subdirectory
RUN npm --prefix frontend install

# Copy the rest of the frontend source code
COPY frontend/ ./frontend/
# Run the build script, which will create the 'frontend/dist' directory
RUN npm --prefix frontend run build

# STAGE 2: Build the Python Production Image
# This stage creates the final, lean container with our Python app and the built frontend.
FROM python:3.13-slim

# Set the final working directory
WORKDIR /app

# Install uv, our fast package manager
RUN pip install uv

# Copy the requirements.txt from the root of our build context
COPY requirements.txt .
# Install the Python dependencies
RUN uv pip install --no-cache-dir --system -r requirements.txt

# Copy the entire backend directory into the container
COPY backend/ ./backend/

# CRITICAL STEP: Copy the built frontend assets from the 'builder' stage.
# The source is the '/app/frontend/dist' directory from Stage 1.
# The destination is './frontend/dist', which matches the exact relative path
# your backend/main.py script expects to find.
COPY --from=builder /app/frontend/dist ./frontend/dist/

# Cloud Run injects a PORT environment variable, which your main.py already uses.
# We expose 8000 as a standard practice.
EXPOSE 8000

# Set the command to run the application.
# We specify the full path to the Python script.
```

```
CMD ["python", "backend/main.py"]
EOF
```

編譯及建構代理程式/前端映像檔

👉 前往包含 Dispatch 代理程式程式碼的後端目錄 (`main.py`)，並將其封裝至容器映像檔。

```
cd $HOME/way-back-home/level_4
export PROJECT_ID=$(gcloud config get-value project)
export REGION=us-central1
export SERVICE_NAME=mission-bravo
export IMAGE_PATH=gcr.io/${PROJECT_ID}/${SERVICE_NAME}
# This assumes your dispatch agent server (main.py) is in the backend folder

gcloud builds submit . --tag ${IMAGE_PATH}
```

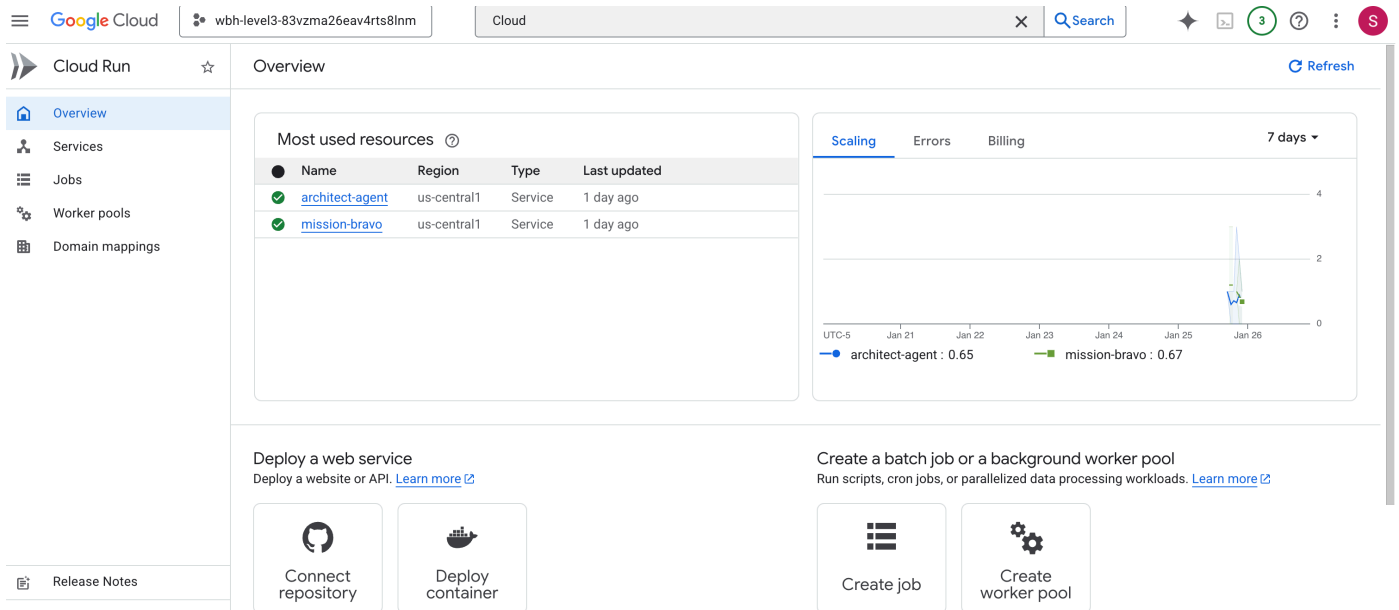
部署至 Cloud Run

👉 將 Dispatch Hub 部署至 Cloud Run。我們會將 **Architect** 的網址注入為環境變數，在兩個雲端原生代理程式之間建立重要連結。

```
export PROJECT_ID=$(gcloud config get-value project)
export REGION=us-central1
export SERVICE_NAME=mission-bravo
export AGENT_SERVICE_NAME=architect-agent
export IMAGE_PATH=gcr.io/${PROJECT_ID}/${SERVICE_NAME}
export PROJECT_NUMBER=$(gcloud projects describe ${PROJECT_ID} --
format="value(projectNumber)")
export
ARCHITECT_AGENT_URL="https://${AGENT_SERVICE_NAME}-${PROJECT_NUMBER}.${REGION}.run.app"
gcloud run deploy ${SERVICE_NAME} \
  --image=${IMAGE_PATH} \
  --platform=managed \
  --region=${REGION} \
  --port=8080 \
  --labels=dev-tutorial=multi-modal \
  --allow-unauthenticated \
  --set-env-vars="ARCHITECT_URL=${ARCHITECT_AGENT_URL}" \
  --set-env-vars="GOOGLE_GENAI_USE_VERTEXAI=True" \
  --set-env-vars="MODEL_ID=gemini-live-2.5-flash-native-audio" \
  --set-env-vars="GOOGLE_CLOUD_PROJECT=${PROJECT_ID}" \
  --set-env-vars="GOOGLE_CLOUD_LOCATION=${REGION}"
```

指令完成後，您會看到服務網址 (例如 `https://mission-bravo-...run.app`)。應用程式現在已在雲端上線。

👉 前往 [Google Cloud Run](#) 頁面，然後從清單中選取 `biometric-scout` 服務。

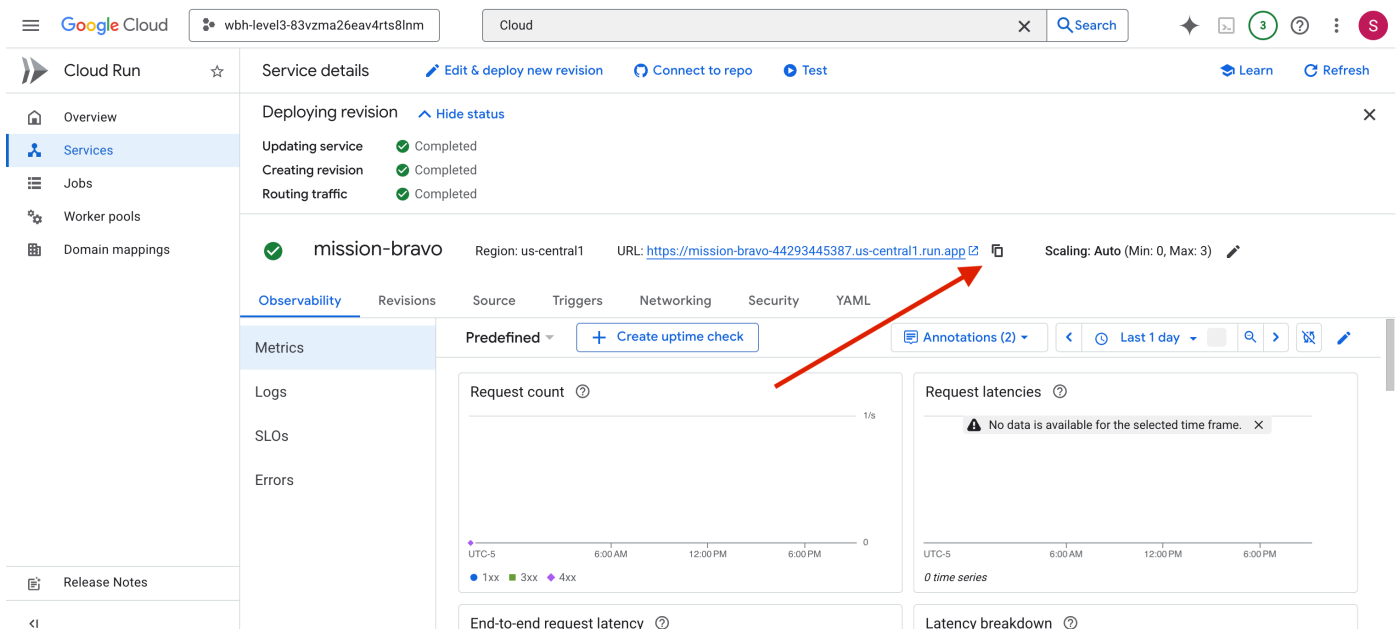


The screenshot shows the Google Cloud Run Overview page. The left sidebar contains navigation links: Overview, Services, Jobs, Worker pools, and Domain mappings. The main content area has a 'Most used resources' table and a 'Scaling' graph.

Name	Region	Type	Last updated
architect-agent	us-central1	Service	1 day ago
mission-bravo	us-central1	Service	1 day ago

The 'Scaling' graph shows CPU usage for 'architect-agent' (0.65) and 'mission-bravo' (0.67) over a 7-day period. Below the graph, there are buttons for 'Connect repository', 'Deploy container', 'Create job', and 'Create worker pool'.

👉 在「服務詳細資料」頁面頂端找到顯示的公開網址。



The screenshot shows the Google Cloud Run Service details page for 'mission-bravo'. The left sidebar contains navigation links: Overview, Services, Jobs, Worker pools, and Domain mappings. The main content area has a 'Deploying revision' section and a 'Request count' graph.

The 'Deploying revision' section shows the following status:

- Updating service: Completed
- Creating revision: Completed
- Routing traffic: Completed

The 'Request count' graph shows the number of requests over time. A red arrow points to the URL: <https://mission-bravo-44293445387.us-central1.run.app>.

最終系統檢查 (端對端測試)

👉 現在你將與即時系統互動。

1. 取得網址：從上一個部署指令的輸出內容中，複製「服務網址」（應以 `run.app` 結尾）。
2. 開啟 **Cockpit**：將網址貼到網路瀏覽器中。
3. 發起聯絡：介面載入後，請務必允許存取螢幕和麥克風。
4. 要求資料：指派硬碟後，要求開始組裝。例如：「開始組裝」



SYSTEM STABLE

/// DRIVE ONLINE ///

PREPARING FOR HYPERSPACE JUMP.

INITIATE JUMP

您現在與完全部署的多代理系統互動，該系統完全在 Google Cloud 上執行。

多重代理系統會將最終的封鎖環鎖定到位，而反常的輻射則會趨於平緩，變成穩定的嗡嗡聲。

「曲速引擎：穩定。救援艇：引擎已啟動。」



在螢幕上，外星船隻向上疾駛，在 Ozymandias 的大氣層崩塌時，險險逃離崩塌的表面。它會與你的船隻一起進入安全軌道，通訊系統中充滿倖存者的聲音，他們雖然受到驚嚇，但仍活著。救援完成後，返家路線暢通無阻，遠端連結就會中

斷。

感謝你，倖存者獲救了。

如果你參加了第 0 級，別忘了查看「返家之路」任務的進度！

